

SOLIDWORKS API (.NET, VBA, C#) — Examples (39/50): Insert_GTol_Example_VB.NET-Insert_Sheet_Metal_Gusset_Feature_Example_VB.NET

SOLIDWORKS API Documentation (converted from CHM)

Insert GTol Example (VB.NET)

This example shows how to insert a GTol with a bent leader.

```

-----
Preconditions:
1. Verify that the drawing exists.
2. Open the Immediate window.
Postconditions:
1. Opens the drawing.
2. Inserts a GTol.
3. Sets and gets the length of the bent leader.
4. Gets whether to display the projected tolerance zone and its height
   in the GTol.
5. Examine the Immediate window and GTol.
NOTE: Because the drawing is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swDrawing As DrawingDoc
        Dim swModel As ModelDoc2
        Dim swGtol As GTol
        Dim swAnno As Annotation
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim frameNbr As Integer
        Dim tolNbr As Integer
        Dim display As Boolean
        Dim height As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swApp.ActivateDoc3("foodprocessor - Sheet1", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors)
        swModel = swDrawing

        swGtol = swModel.InsertGTol()
        If Not swGtol Is Nothing Then
            swGtol.SetFrameSymbols2(1, "<IGTOL-POSI>", False, "", False, "", "", "", "")
            status = swGtol.SetFrameValues2(1, "0.2", "", "B-A-C<MOD-MMC>", "B<MOD-MMC>-C<MOD-LMC>", "C<MOD-MMC>-A")
            status = swGtol.SetPTZHeight2(1, 1, True, "50")
            swGtol.SetBetweenTwoPoints(False, "", "")
            swAnno = swGtol.GetAnnotation()
            If Not swAnno Is Nothing Then
                status = swAnno.SetPosition(0.319315975204224, 0.12666668401487, 0)
                errors = swAnno.SetLeader3(swLeaderStyle_e.swBENT, swLeaderSide_e.swLS_LEFT, True, False, False, False)
                swAnno.BentLeaderLength = 0.05
                Debug.Print("Bent leader length: " & swAnno.BentLeaderLength)
            End If
        End If

        frameNbr = 1
        tolNbr = 1
        height = ""
        status = swGtol.GetPTZHeight2(frameNbr, tolNbr, display, height)
        Debug.Print("Projected tolerance zone: ")
        Debug.Print("  Display: " & display)
        Debug.Print("  Height: " & height)

        swModel.WindowRedraw()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert Gusset Feature Example (C#)

This example shows how to create a weldment gusset feature.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
//
// Postconditions:
// 1. Creates Gusset5.
// 2. Expand CutList(32) in the FeatureManager design tree and
//    scroll to the bottom of the list.
// 3. Examine the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertGusset_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;
        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID("", "FACE", 0.600909534718824, 0.15614125327869, -0.9850000000000127, true, 1,
null, 0);
            boolstatus = Part.Extension.SelectByID("", "FACE", 0.597259667129663, 0.01499999999998727, -0.944257845206607, true, 1,
null, 0);

            myFeature = Part.FeatureManager.InsertGussetFeature3(0.005, (int)swGussetThicknessType_e.swGussetThicknessBothSides,
(int)swGussetProfileLocationType_e.swGussetProfileLocationCenter, false, 0.025, 0.025, 0.015, 0.78539816339745, 0.015, false,
0.005, 0, false, false, false, 0.0125, 0.0125, 0.785398163397452, true, false);
        }

        public SldWorks swApp;
    }
}
```

Insert Gusset Feature Example (VBA)

This example shows how to create a weldment gusset feature.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\weldment_box3.sldprt.  
'  
' Postconditions:  
' 1. Creates Gusset5.  
' 2. Expand CutList(32) in the FeatureManager design tree and  
'    scroll to the bottom of the list.  
' 3. Examine the graphics area.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim myFeature As SldWorks.Feature  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
  
    boolstatus = Part.Extension.SelectByID2("", "FACE", 0.600909534718824, 0.15614125327869, -0.985000000000127, True, 1, Nothing,  
0)  
    boolstatus = Part.Extension.SelectByID2("", "FACE", 0.597259667129663, 1.49999999998727E-02, -0.944257845206607, True, 1,  
Nothing, 0)  
  
    Set myFeature = Part.FeatureManager.InsertGussetFeature3(0.005, swGussetThicknessType_e.swGussetThicknessBothSides,  
swGussetProfileLocationType_e.swGussetProfileLocationCenter, False, 0.025, 0.025, 0.015, 0.78539816339745, 0.015, False, 0.005, 0,  
False, False, False, 0.0125, 0.0125, 0.785398163397452, True, False)  
  
End Sub
```

Insert Gusset Feature Example (VB.NET)

This example shows how to create a weldment gusset feature.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
'
' Postconditions:
' 1. Creates Gusset5.
' 2. Expand CutLit(32) in the FeatureManager design tree and
'    scroll to the bottom of the list.
' 3. Examine the graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim myFeature As Feature
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
```

```
        boolstatus = Part.Extension.SelectByID("", "FACE", 0.600909534718824, 0.15614125327869, -0.985000000000127, True, 1,
Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID("", "FACE", 0.597259667129663, 0.01499999999998727, -0.944257845206607, True, 1,
Nothing, 0)
```

```
        myFeature = Part.FeatureManager.InsertGussetFeature3(0.005, swGussetThicknessType_e.swGussetThicknessBothSides,
swGussetProfileLocationType_e.swGussetProfileLocationCenter, False, 0.025, 0.025, 0.015, 0.78539816339745, 0.015, False, 0.005, 0,
False, False, False, 0.0125, 0.0125, 0.785398163397452, True, False)
```

```
    End Sub
```

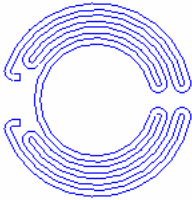
```
    Public swApp As SldWorks
```

```
End Class
```

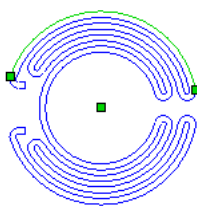
Insert Hatch Example (VBA)

This example shows how to insert a hatch on a closed sketch in a drawing. The following image demonstrates the example.

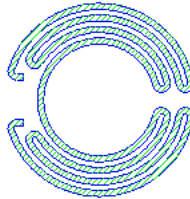
Before selection



Arc29 selected



Hatch inserted



```

'-----
' Preconditions:
' 1. Open a drawing containing a closed sketch similar to the
' sketch shown.
' 2. Select an arc.
' 3. Change the name of the arc in this code to match name of
' of the arc selected in step 2 (e.g., record selecting the
' arc, save the recording to a new macro, open and inspect
' that macro, and make note of the name of the selected arc).
'
' Postconditions:
' 1. Inserts and scales the hatch.
' 2. Examines the drawing.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim SelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    Set SelMgr = Part.SelectionManager
    boolstatus = Part.Extension.SelectByID2("Arc29", "SKETCHSEGMENT", 0.08421725979537, 0.08635799134766, 0, False, 0, Nothing, 0)

    Dim selSkSeg As SldWorks.SketchSegment
    Dim selSk As SldWorks.Sketch
    Set selSkSeg = SelMgr.GetSelectedObject6(1, -1)
    Set selSk = selSkSeg.GetSketch

    Part.InsertHatchedFace

    Dim hatchArr As Variant
    Dim vobj As Variant
    Dim skHatch As SldWorks.SketchHatch

    hatchArr = selSk.GetSketchHatches
    For Each vobj In hatchArr
        Set skHatch = vobj
        skHatch.Scale2 = 4
    Next vobj
End Sub

```

Insert Hole Table Example (C#)

This example shows how to insert a hole table in a drawing.

```
//-----
// Preconditions: Ensure that the specified part to open, drawing template,
// and hole table template exist.
//
// Postconditions:
// 1. Opens the part and creates a drawing of it.
// 2. Inserts a hole table of the part in the drawing.
// 3. Examine the hole table in the drawing.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DrawingDoc Drawing;
        bool boolstatus;

        public void Main()
        {
            DocumentSpecification spec = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2019\\samples\\tutorial\\api\\cover_datum.sldprt");
            Part = (ModelDoc2)swApp.OpenDoc7(spec);
            Drawing = (DrawingDoc)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2019\\templates\\Drawing.drwdot", 2, 0.2794, 0.4318);
            Part = (ModelDoc2)Drawing;
            boolstatus = Part.Extension.SelectByID2("Sheet1", "SHEET", 0.39237, 0.5218942019544, 0, false, 0, null, 0);
            boolstatus = Drawing.Create3rdAngleViews2("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2019\\samples\\tutorial\\api\\cover_datum.sldprt");
            Part.ClearSelection2(true);

            boolstatus = Drawing.ActivateView("Drawing View1");

            //Select a vertex in the drawing view to be the origin of all datums in the table
            //All XLOC and YLOC table column values will be relative to this datum origin
            boolstatus = Part.Extension.SelectByID2("", "VERTEX", 0.117324728174898, 0.108554228680764, -500.0075, true, 1, null, 0);
            //Select a face that contains the holes that will be annotated in the table
            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.090728339186173, 0.119052803281577, -500.0075, true, 2, null, 0);

            View myView = null;
            SelectionMgr selMgr = null;
            selMgr = (SelectionMgr)Part.SelectionManager;
            myView = selMgr.GetSelectedObjectsDrawingView2(1, -1);
            HoleTableAnnotation myHoleTable = null;
            //Insert a hole table
            //anchored with its top left corner at x-coordinate = 0.07m and y-coordinate = 0.175m,
            //with starting datum tag "A",
            //using hole table template: standard hole table--letters.sldholtbt
            myHoleTable = (HoleTableAnnotation)myView.InsertHoleTable3(false, 0.153019881817662, -3.77259107537343E-
02, (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "A", "C:\\Program Files\\SolidWorks Corp\\SOLIDWORKS\\lang\\english\\standard ho
le table--letters.sldholtbt", 1, 1, null);

            Part.ClearSelection2(true);

            boolstatus = Drawing.ActivateSheet("Sheet1");
        }
    }
}
public SldWorks swApp;
```

Insert Hole Table Example (VBA)

This example shows how to insert a hole table in a drawing.

```

'-----
' Preconditions: Ensure that the specified part to open, drawing template,
' and hole table template exist.
'
' Postconditions:
' 1. Opens the part and creates a drawing of it.
' 2. Inserts a hole table of the part in the drawing.
' 3. Examine the hole table in the drawing.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim spec As SldWorks.DocumentSpecification
Dim Drawing As SldWorks.DrawingDoc
Dim boolstatus As Boolean

Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set spec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\cover_datum.sldprt")
    Set Part = swApp.OpenDoc7(spec)
    Set Drawing = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2019\templates\Drawing.drwdot", 2, 0.2794, 0.4318)

    Set Part = Drawing

    boolstatus = Part.Extension.SelectByID2("Sheet1", "SHEET", 0.39237, 0.5218942019544, 0, False, 0, Nothing, 0)
    boolstatus = Part.Create3rdAngleViews2("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\cover_datum.sldprt")
    Part.ClearSelection2 True

    boolstatus = Part.ActivateView("Drawing View1")

    'Select a vertex in the drawing view to be the origin of all datums in the table
    'All XLOC and YLOC table column values will be relative to this datum origin
    boolstatus = Part.Extension.SelectByID2("", "VERTEX", 0.117324728174898, 0.108554228680764, -500.0075, True, 1, Nothing, 0)
    'Select a face that contains the holes that will be annotated in the table
    boolstatus = Part.Extension.SelectByID2("", "FACE", 0.090728339186173, 0.119052803281577, -500.0075, True, 2, Nothing, 0)

    Dim myView As SldWorks.View
    Set myView = Part.SelectionManager.GetSelectedObjectsDrawingView2(1, -1)
    Dim myHoleTable As SldWorks.HoleTableAnnotation
    'Insert a hole table
    'anchored with its top left corner at x-coordinate = 0.07m and y-coordinate = 0.175m,
    'with starting datum tag "A",
    'using hole table template: standard hole table--letters.sldholtbt
    Set myHoleTable = myView.InsertHoleTable3(False, 0.153019881817662, -3.77259107537343E-02, 1, "A", "C:\Program Files\SolidWorks
Corp\SOLIDWORKS\lang\english\standard hole table--letters.sldholtbt", 1, 1, Nothing)

    Part.ClearSelection2 True

    boolstatus = Part.ActivateSheet("Sheet1")

End Sub

```

Insert Hole Table Example (VB.NET)

This example shows how to insert a hole table in a drawing.

```

'-----
' Preconditions: Ensure that the specified part to open, drawing template,
' and hole table template exist.
'
' Postconditions:
' 1. Opens the part and creates a drawing of it.
' 2. Inserts a hole table of the part in the drawing.
' 3. Examine the hole table in the drawing.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim Drawing As DrawingDoc
    Dim spec As DocumentSpecification
    Dim boolstatus As Boolean

    Sub main()

        spec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\cover_datum.sldprt")
        Part = swApp.OpenDoc7(spec)
        Drawing = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2019\templates\Drawing.drwdot", 2, 0.2794, 0.4318)

        Part = Drawing

        boolstatus = Part.Extension.SelectByID2("Sheet1", "SHEET", 0.39237, 0.5218942019544, 0, False, 0, Nothing, 0)
        boolstatus = Part.Create3rdAngleViews2("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\cover_datum.sldprt")
        Part.ClearSelection2(True)

        boolstatus = Part.ActivateView("Drawing View1")

        'Select a vertex in the drawing view to be the origin of all datums in the table
        'All XLOC and YLOC table column values will be relative to this datum origin
        boolstatus = Part.Extension.SelectByID2("", "VERTEX", 0.117324728174898, 0.108554228680764, -500.0075, True, 1, Nothing, 0)
        'Select a face that contains the holes that will be annotated in the table
        boolstatus = Part.Extension.SelectByID2("", "FACE", 0.090728339186173, 0.119052803281577, -500.0075, True, 2, Nothing, 0)
        Dim myView As View
        myView = Part.SelectionManager.GetSelectedObjectsDrawingView2(1, -1)
        Dim myHoleTable As HoleTableAnnotation
        'Insert a hole table
        'anchored with its top left corner at x-coordinate = 0.07m and y-coordinate = 0.175m,
        'with starting datum tag "A",
        'using hole table template: standard hole table--letters.sldholtbt
        myHoleTable = myView.InsertHoleTable3(False, 0.153019881817662, -0.0377259107537343, 1, "A", "C:\Program Files\SolidWorks Corp\SOLIDWORKS\lang\english\standard hole table--letters.sldholtbt", 1, 1, Nothing)

        Part.ClearSelection2(True)

        boolstatus = Part.ActivateSheet("Sheet1")
    End Sub

    Public swApp As SldWorks

End Class

```

Insert Hole Wizard Slot and Hole Example (C#)

This example shows how to use [IFeatureManager.HoleWizard5](#) to insert a straight slot and a counterbore hole in a part.

```
//-----
// Preconditions:
// 1. Verify that the specified part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Creates a straight slot and a counterbore hole.
// 3. Prints the length of the slot to the
//    Immediate window.
// 4. To verify steps 2 and 3, examine the part in the
//    graphics area, the FeatureManager design tree, and
//    the Immediate window.
//
// NOTE: Because the part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

Namespace HoleWizard5FeatureManagerCSharp.csproj
{
    Partial Class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatureMgr = default(FeatureManager);
            Feature swFeature = default(Feature);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool status = false;
            int SlotType = 0;
            int HoleType = 0;
            int StandardIndex = 0;
            int FastenerTypeIndex = 0;
            string SSize = null;
            short EndType = 0;
            double ConvFactorLength = 0;
            double ConvFactorAngle = 0;
            double Diameter = 0;
            double Depth = 0;
            double Length = 0;
            double ScrewFit = 0;
            double DrillAngle = 0;
            double NearCsinkDiameter = 0;
            double NearCsinkAngle = 0;
            double FarCsinkDiameter = 0;
            double FarCsinkAngle = 0;
            double Offset = 0;
            string ThreadClass = null;
            double CounterBoreDiameter = 0;
            double CounterBoreDepth = 0;
            double HeadClearance = 0;
            double BotCsinkDiameter = 0;
            double BotCsinkAngle = 0;
            WizardHoleFeatureData2 swWizardHoleFeatData = default(WizardHoleFeatureData2);

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2020\\samples\\tutorial\\api\\block20.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Create a slot

            //Select the face where to create the slot
            status = swModelDocExt.SelectByID("", "FACE", -0.000609805077203873, 0.0396239999998897, -0.0083087834611201, false, 0, null, 0);

            SlotType = (int)swWzdGeneralHoleTypes_e.swWzdHoleSlot;
            StandardIndex = (int)swWzdHoleStandards_e.swStandardAnsiInch;
            FastenerTypeIndex = (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchAllDrillSizes;
            SSize = "#97";
            EndType = (int)swEndConditions_e.swEndCondBlind;
            ConvFactorLength = 25.4 / 1000; //Convert inches to meters
            ConvFactorAngle = (22 / 7) / 180; //Convert degrees to radians
            Diameter = 0.5 * ConvFactorLength;
            Depth = 2 * ConvFactorLength;
            Length = 3 * ConvFactorLength;

            //Value1 to Value12 arguments; SOLIDWORKS Design
            //ignores parameters set to -1
            ScrewFit = -1; //Value1
            DrillAngle = 100 * ConvFactorAngle; //Value2
            NearCsinkDiameter = -1; //Value3
            NearCsinkAngle = -1; //Value4
            FarCsinkDiameter = -1; //Value5
            FarCsinkAngle = -1; //Value6
            Offset = -1; //Value7
            //Value8 - Value12 all set to -1
        }
    }
}
```

```

ThreadClass = "";

swFeature = (Feature)swFeatureMgr.HoleWizard5(SlotType, StandardIndex, FastenerTypeIndex, SSize, EndType, Diameter, Depth, Length,
ScrewFit, DrillAngle, NearCsinkDiameter, NearCsinkAngle, FarCsinkDiameter, FarCsinkAngle, Offset, -1, -1, -1, -1, -1,
ThreadClass, false, false, false, false, false, false);

//Print length of slot to Immediate window
swWizardHoleFeatData = (WizardHoleFeatureData2)swFeature.GetDefinition();
Debug.Print("Length of slot: " + swWizardHoleFeatData.Length + " inches");

//Create a counterbore hole

//Select the face where to create the hole
status = swModelDocExt.SelectByID2("", "FACE", -0.0060197480091233, 0.0396239999998329, 0.0270812377555103, false, 0, null, 0);

HoleType = (int)swWzdGeneralHoleTypes_e.swWzdCounterBore;
StandardIndex = (int)swWzdHoleStandards_e.swStandardAnsiInch;
FastenerTypeIndex = (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBinding;
SSize = "#12";
EndType = (int)swEndConditions_e.swEndCondThroughAll;
ConvFactorLength = 25.4 / 1000; //Convert inches to meters
ConvFactorAngle = (22 / 7) / 180; //Convert degrees to radians
Diameter = 0.5 * ConvFactorLength;
Depth = -1;
Length = -1;

//Value1 to Value12 arguments; SOLIDWORKS Design
//ignores parameters set to -1
CounterBoreDiameter = 0.6 * ConvFactorLength; //Value1
CounterBoreDepth = 0.2 * ConvFactorLength; //Value2
HeadClearance = -1; //Value3
ScrewFit = -1; //Value4
DrillAngle = -1; //Value5
NearCsinkDiameter = -1; //Value6
NearCsinkAngle = -1; //Value7
BotCsinkDiameter = -1; //Value8
BotCsinkAngle = -1; //Value9
FarCsinkDiameter = -1; //Value10
FarCsinkAngle = -1; //Value11
Offset = -1; //Value12

ThreadClass = "";

swFeature = (Feature)swFeatureMgr.HoleWizard5(HoleType, StandardIndex, FastenerTypeIndex, SSize, EndType,
Diameter, Depth, Length, CounterBoreDiameter, CounterBoreDepth, HeadClearance, ScrewFit, DrillAngle,
NearCsinkDiameter, NearCsinkAngle, BotCsinkDiameter, BotCsinkAngle, FarCsinkDiameter, FarCsinkAngle,
Offset, ThreadClass, false, false, false, false, false, false);

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;

}
}

```

Insert Hole Wizard Slot and Hole Example (VBA)

This example shows how to use [IFeatureManager.HoleWizard5](#) to insert a straight slot and a counterbore hole in a part.

```
'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Creates a straight slot and a counterbore hole.
' 3. Prints the length of the slot to the Immediate
'    window.
' 4. To verify steps 2 and 3, examine the part
'    in the graphics area, the FeatureManager
'    design tree, and the Immediate window.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim status As Boolean
Dim SlotType As Long
Dim HoleType As Long
Dim StandardIndex As Long
Dim FastenerTypeIndex As Long
Dim SSize As String
Dim EndType As Long
Dim ConvFactorLength As Double
Dim ConvFactorAngle As Double
Dim Diameter As Double
Dim Depth As Double
Dim Length As Double
Dim ScrewFit As Double
Dim DrillAngle As Double
Dim NearCsinkDiameter As Double
Dim NearCsinkAngle As Double
Dim FarCsinkDiameter As Double
Dim FarCsinkAngle As Double
Dim Offset As Double
Dim ThreadClass As String
Dim CounterBoreDiameter As Double
Dim CounterBoreDepth As Double
Dim HeadClearance As Double
Dim BotCsinkDiameter As Double
Dim BotCsinkAngle As Double
Dim swWizardHoleFeatData As SldWorks.WizardHoleFeatureData2

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\block20.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swFeatureMgr = swModel.FeatureManager
    Set swModelDocExt = swModel.Extension

    'Create a slot

    'Select the face where to create the slot
    status = swModelDocExt.SelectByID("", "FACE", -6.09805077203873E-04, 3.96239999998897E-02, -8.30387834611201E-03, False, 0, Nothing, 0)

    SlotType = swWzdGeneralHoleTypes_e.swWzdHoleSlot
    StandardIndex = swWzdHoleStandards_e.swStandardAnsiInch
    FastenerTypeIndex = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchAllDrillSizes
    SSize = "#97"
    EndType = swEndConditions_e.swEndCondBlind
    ConvFactorLength = 25.4 / 1000 'Convert inches to meters
    ConvFactorAngle = (22 / 7) / 180 'Convert degrees to radians
    Diameter = 0.5 * ConvFactorLength
    Depth = 2 * ConvFactorLength
    Length = 3 * ConvFactorLength

    'Value1 to Value7 arguments; SOLIDWORKS Design ignores parameters set to -1

    ScrewFit = -1 'Value1
    DrillAngle = 100 * ConvFactorAngle 'Value2
    NearCsinkDiameter = -1 'Value3
    NearCsinkAngle = -1 'Value4
    FarCsinkDiameter = -1 'Value5
    FarCsinkAngle = -1 'Value6
    Offset = -1 'Value7
    'Value8 - Value12 are all -1

    ThreadClass = ""
```

```

Set swFeature = swFeatureMgr.HoleWizard5(SlotType, StandardIndex, FastenerTypeIndex, _
    SSize, EndType, Diameter, Depth, Length, ScrewFit, DrillAngle, _
    NearCsinkDiameter, NearCsinkAngle, FarCsinkDiameter, FarCsinkAngle, Offset, -1, -1, -1, _
    -1, -1, ThreadClass, False, False, False, False, False, False)

'Print length of slot to Immediate window
Set swWizardHoleFeatData = swFeature.GetDefinition
Debug.Print "Length of slot: " & swWizardHoleFeatData.Length & " inches"

'Create a counterbore hole

'Select the face where to create the hole
status = swModelDocExt.SelectByID2("", "FACE", -6.0197480091233E-03, 3.96239999998329E-02, 2.70812377555103E-02, False, 0, Nothing, 0)

HoleType = swWzdGeneralHoleTypes_e.swWzdCounterBore
StandardIndex = swWzdHoleStandards_e.swStandardAnsiInch
FastenerTypeIndex = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBinding
SSize = "#12"
EndType = swEndConditions_e.swEndCondThroughAll
ConvFactorLength = 25.4 / 1000 'Convert inches to meters
ConvFactorAngle = (22 / 7) / 180 'Convert degrees to radians
Diameter = 0.5 * ConvFactorLength
Depth = -1
Length = -1 'Holes do not have length

'Value1 to Value12 arguments; SOLIDWORKS Design ignores parameters set to -1
CounterBoreDiameter = 0.6 * ConvFactorLength 'Value1
CounterBoreDepth = 0.2 * ConvFactorLength 'Value2
HeadClearance = -1 'Value3
ScrewFit = -1 'Value4
DrillAngle = -1 'Value5
NearCsinkDiameter = -1 'Value6
NearCsinkAngle = -1 'Value7
BotCsinkDiameter = -1 'Value8
BotCsinkAngle = -1 'Value9
FarCsinkDiameter = -1 'Value10
FarCsinkAngle = -1 'Value11
Offset = -1 'Value12

ThreadClass = ""

Set swFeature = swFeatureMgr.HoleWizard5(HoleType, StandardIndex, FastenerTypeIndex, SSize, EndType, _
    Diameter, Depth, Length, CounterBoreDiameter, CounterBoreDepth, HeadClearance, ScrewFit, DrillAngle, _
    NearCsinkDiameter, NearCsinkAngle, BotCsinkDiameter, BotCsinkAngle, FarCsinkDiameter, FarCsinkAngle, _
    Offset, ThreadClass, False, False, False, False, False, False)

```

End Sub

Insert Hole Wizard Slot and Hole Example (VB.NET)

This example shows how to use [IFeatureManager.HoleWizard5](#) to insert a straight slot and a counterbore hole in a part.

```

-----
Preconditions:
1. Verify that the specified part document to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified part document.
2. Creates a straight slot and a counterbore hole.
3. Prints the length of the slot to the Immediate window.
4. To verify steps 2 and 3, examine the part in the graphics area, the FeatureManager design tree, and the Immediate window.

NOTE: Because the part document is used elsewhere, do not save changes.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatureMgr As FeatureManager
        Dim swFeature As Feature
        Dim swModelDocExt As ModelDocExtension
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim status As Boolean
        Dim SlotType As Integer
        Dim HoleType As Integer
        Dim StandardIndex As Integer
        Dim FastenerTypeIndex As Integer
        Dim SSize As String
        Dim EndType As Integer
        Dim ConvFactorLength As Double
        Dim ConvFactorAngle As Double
        Dim Diameter As Double
        Dim Depth As Double
        Dim Length As Double
        Dim ScrewFit As Double
        Dim DrillAngle As Double
        Dim NearCsinkDiameter As Double
        Dim NearCsinkAngle As Double
        Dim FarCsinkDiameter As Double
        Dim FarCsinkAngle As Double
        Dim Offset As Double
        Dim ThreadClass As String
        Dim CounterBoreDiameter As Double
        Dim CounterBoreDepth As Double
        Dim HeadClearance As Double
        Dim BotCsinkDiameter As Double
        Dim BotCsinkAngle As Double
        Dim swWizardHoleFeatData As WizardHoleFeatureData2

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\block20.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swFeatureMgr = swModel.FeatureManager
        swModelDocExt = swModel.Extension

        'Create a slot

        'Select the face where to create the slot
        status = swModelDocExt.SelectByID("", "FACE", -0.000609805077203873, 0.03962399999998897, -0.00830387834611201, False, 0, Nothing, 0)

        SlotType = swWzdGeneralHoleTypes_e.swWzdHoleSlot
        StandardIndex = swWzdHoleStandards_e.swStandardAnsiInch
        FastenerTypeIndex = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchAllDrillSizes
        SSize = "#97"
        EndType = swEndConditions_e.swEndCondBlind
        ConvFactorLength = 25.4 / 1000 'Convert inches to meters
        ConvFactorAngle = (22 / 7) / 180 'Convert degrees to radians
        Diameter = 0.5 * ConvFactorLength
        Depth = 2 * ConvFactorLength
        Length = 3 * ConvFactorLength

        'Value1 to Value12 arguments; SOLIDWORKS Design
        'ignores parameters set to -1
        ScrewFit = -1 'Value1
        DrillAngle = 100 * ConvFactorAngle 'Value2
        NearCsinkDiameter = -1 'Value3
        NearCsinkAngle = -1 'Value4
        FarCsinkDiameter = -1 'Value5
        FarCsinkAngle = -1 'Value6
        Offset = -1 'Value7
        'Value8 - Value12 all set to -1

        ThreadClass = ""

        swFeature = swFeatureMgr.HoleWizard5(SlotType, StandardIndex, FastenerTypeIndex, SSize, EndType, _
            Diameter, Depth, Length, ScrewFit, DrillAngle, NearCsinkDiameter, NearCsinkAngle, _
            FarCsinkDiameter, FarCsinkAngle, Offset, -1, -1, -1, -1, -1, ThreadClass, _
            False, False, False, False, False, False)

        'Print length of slot to Immediate window
        swWizardHoleFeatData = swFeature.GetDefinition
        Debug.Print("Length of slot: " & swWizardHoleFeatData.Length & " inches")

        'Create a counterbore hole
    
```

```
'Select the face where to create the hole
status = swModelDocExt.SelectByID("", "FACE", -0.0060197480091233, 0.0396239999998329, 0.0270812377555103, False, 0, Nothing, 0)
```

```
HoleType = swWzdGeneralHoleTypes_e.swWzdCounterBore
StandardIndex = swWzdHoleStandards_e.swStandardAnsiInch
FastenerTypeIndex = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBinding
SSize = "#12"
EndType = swEndConditions_e.swEndCondThroughAll
ConvFactorLength = 25.4 / 1000 'Convert inches to meters
ConvFactorAngle = (22 / 7) / 180 'Convert degrees to radians
Diameter = 0.5 * ConvFactorLength
Depth = -1
Length = -1
```

```
'Value1 to Value12 arguments; SOLIDWORKS Design
'ignores parameters set to -1
CounterBoreDiameter = 0.6 * ConvFactorLength 'Value1
CounterBoreDepth = 0.2 * ConvFactorLength 'Value2
HeadClearance = -1 'Value3
ScrewFit = -1 'Value4
DrillAngle = -1 'Value5
NearCsinkDiameter = -1 'Value6
NearCsinkAngle = -1 'Value7
BotCsinkDiameter = -1 'Value8
BotCsinkAngle = -1 'Value9
FarCsinkDiameter = -1 'Value10
FarCsinkAngle = -1 'Value11
Offset = -1 'Value12
```

```
ThreadClass = ""
```

```
swFeature = swFeatureMgr.HoleWizard5(HoleType, StandardIndex, FastenerTypeIndex, SSize, EndType,
Diameter, Depth, Length, CounterBoreDiameter, CounterBoreDepth, HeadClearance, ScrewFit, DrillAngle,
NearCsinkDiameter, NearCsinkAngle, BotCsinkDiameter, BotCsinkAngle, FarCsinkDiameter, FarCsinkAngle, Offset,
ThreadClass, False, False, False, False, False, False)
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Insert Indent Feature Example (C#)

This example shows how to insert and modify an indent feature.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Selects the boss-extrude body and a face on the
//    extrude-thin body.
// 3. Inserts an indent feature.
// 4. Modifies the thickness of the indent feature.
// 5. Examine the Immediate window, FeatureManager design tree,
//    and graphics area.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureMgr = default(FeatureManager);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            IndentFeatureData swIndentFeatureData = default(IndentFeatureData);
            Body2 targetBody = default(Body2);
            Face2 swFace = default(Face2);
            Body2 toolRegionBody = default(Body2);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            object[] toolBodyRegions = null;
            int toolBodyRegionType = 0;
            int nbrBodies = 0;
            int i = 0;

            //Open part where to insert indent feature
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Select solid body for target body,
            //select face for tool body region, and
            //and insert indent feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0371343422566497, -0.01499999999999864, 0.0883053842352979, true, 4, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.InsertIndent(0.01, 0, true, true, false, false);

            //Access and modify indent feature
            Debug.Print("Indent feature name: " + swFeature.Name);
            swIndentFeatureData = (IndentFeatureData)swFeature.GetDefinition();
            swIndentFeatureData.AccessSelections(swModel, null);
            nbrBodies = swIndentFeatureData.GetBodiesCount();
            Debug.Print("  Number of bodies: " + nbrBodies);
            targetBody = (Body2)swIndentFeatureData.TargetBody;
            Debug.Print("  Name of target body: " + targetBody.Name);
            toolBodyRegions = (object[])swIndentFeatureData.ToolBodyRegion;
            Debug.Print("  Number of tool body regions: " + toolBodyRegions.Length);
            for (i = 0; i < nbrBodies; i++)
            {
                swModel.ClearSelection2(true);
                swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
                status = swSelectionMgr.AddSelectionListObject(toolBodyRegions[i], null);
                toolBodyRegionType = swSelectionMgr.GetSelectedObjectType3(1, -1);
                Debug.Print("  Type of object selected for tool body region (2 = face; 3 = vertex): " + toolBodyRegionType);
                //If object selected for tool body region is a face,
                //then get the name of its body
                if (toolBodyRegionType == 2)
                {
                    swFace = (Face2)toolBodyRegions[i];
                    toolRegionBody = (Body2)swFace.GetBody();
                    Debug.Print("  Name of body of tool body region: " + toolRegionBody.Name);
                }
            }
            Debug.Print("  Original thickness: " + swIndentFeatureData.Thickness);
            //Change thickness
        }
    }
}
```

```
swIndentFeatureData.Thickness = 0.011;
Debug.Print(" Modified thickness: " + swIndentFeatureData.Thickness);
status = swFeature.ModifyDefinition(swIndentFeatureData, swModel, null);

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
```

Insert Indent Feature Example (VBA)

This example shows how to insert and modify an indent feature.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Selects the boss-extrude body and a face on the
'    extrude-thin body.
' 3. Inserts an indent feature.
' 4. Modifies the thickness of the indent feature.
' 5. Examine the Immediate window, FeatureManager design tree,
'    and graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swIndentFeatureData As SldWorks.IndentFeatureData
Dim targetBody As SldWorks.Body2
Dim swFace As SldWorks.Face2
Dim toolRegionBody As SldWorks.Body2
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim toolBodyRegions As Variant
Dim toolBodyRegionType As Long
Dim nbrBodies As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open part where to insert indent feature
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select solid body for target body,
    'select face for tool body region, and
    'and insert indent feature
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "FACE", -3.71343422566497E-02, -1.49999999999864E-02, 8.83053842352979E-02, True, 4, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.InsertIndent(0.01, 0, True, True, False, False)

    'Access and modify indent feature
    Debug.Print "Indent feature name: " & swFeature.Name
    Set swIndentFeatureData = swFeature.GetDefinition
    swIndentFeatureData.AccessSelections swModel, Nothing
    nbrBodies = swIndentFeatureData.GetBodiesCount
    Debug.Print " Number of bodies: " & nbrBodies
    Set targetBody = swIndentFeatureData.targetBody
    Debug.Print " Name of target body: " & targetBody.Name
    toolBodyRegions = swIndentFeatureData.toolBodyRegion
    Debug.Print " Number of tool body regions: " & UBound(toolBodyRegions) + 1
    For i = 0 To nbrBodies - 1
        swModel.ClearSelection2 True
        Set swSelectionMgr = swModel.SelectionManager
        status = swSelectionMgr.AddSelectionListObject(toolBodyRegions(i), Nothing)
        toolBodyRegionType = swSelectionMgr.GetSelectedObjectType3(1, -1)
        Debug.Print " Type of object selected for tool body region (2 = face; 3 = vertex): " & toolBodyRegionType
        'If object selected for tool body region is a face,
        'then get the name of its body
        If toolBodyRegionType = 2 Then
            Set swFace = toolBodyRegions(i)
            Set toolRegionBody = swFace.GetBody
            Debug.Print " Name of body of tool body region: " & toolRegionBody.Name
        End If
    Next i
    Debug.Print " Original thickness: " & swIndentFeatureData.Thickness
    'Change thickness
    swIndentFeatureData.Thickness = 0.011
    Debug.Print " Modified thickness: " & swIndentFeatureData.Thickness
    status = swFeature.ModifyDefinition(swIndentFeatureData, swModel, Nothing)

End Sub

```

Insert Indent Feature Example (VB.NET)

This example shows how to insert and modify an indent feature.

```

-----
Preconditions:
1. Verify that the part to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified part.
2. Selects the boss-extrude body and a face on the
   extrude-thin body.
3. Inserts an indent feature.
4. Modifies the thickness of the indent feature.
5. Examine the Immediate window, FeatureManager design tree,
   and graphics area.

NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureMgr As FeatureManager
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swIndentFeatureData As IndentFeatureData
        Dim targetBody As Body2
        Dim swFace As Face2
        Dim toolRegionBody As Body2
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim toolBodyRegions() As Object
        Dim toolBodyRegionType As Integer
        Dim nbrBodies As Integer
        Dim i As Integer

        'Open part where to insert indent feature
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select solid body for target body,
        'select face for tool body region, and
        'and insert indent feature
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", -0.0371343422566497, -0.01499999999999864, 0.0883053842352979, True, 4, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.InsertIndent(0.01, 0, True, True, False, False)

        'Access and modify indent feature
        Debug.Print("Indent feature name: " & swFeature.Name)
        swIndentFeatureData = swFeature.GetDefinition
        swIndentFeatureData.AccessSelections(swModel, Nothing)
        nbrBodies = swIndentFeatureData.GetBodiesCount
        Debug.Print(" Number of bodies: " & nbrBodies)
        targetBody = swIndentFeatureData.TargetBody
        Debug.Print(" Name of target body: " & targetBody.Name)
        toolBodyRegions = swIndentFeatureData.ToolBodyRegion
        Debug.Print(" Number of tool body regions: " & UBound(toolBodyRegions) + 1)
        For i = 0 To nbrBodies - 1
            swModel.ClearSelection2(True)
            swSelectionMgr = swModel.SelectionManager
            status = swSelectionMgr.AddSelectionListObject(toolBodyRegions(i), Nothing)
            toolBodyRegionType = swSelectionMgr.GetSelectedObjectType3(1, -1)
            Debug.Print(" Type of object selected for tool body region (2 = face; 3 = vertex): " & toolBodyRegionType)
            'If object selected for tool body region is a face,
            'then get the name of its body
            If toolBodyRegionType = 2 Then
                swFace = toolBodyRegions(i)
                toolRegionBody = swFace.GetBody
                Debug.Print(" Name of body of tool body region: " & toolRegionBody.Name)
            End If
        Next i
        Debug.Print(" Original thickness: " & swIndentFeatureData.Thickness)
        'Change thickness
        swIndentFeatureData.Thickness = 0.011
        Debug.Print(" Modified thickness: " & swIndentFeatureData.Thickness)
        status = swFeature.ModifyDefinition(swIndentFeatureData, swModel, Nothing)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Insert Jagged Cut Break Example (C#)

This example shows how to insert a jagged cut break in a drawing view.

```
//-----
// Preconditions:
// 1. Verify that the drawing to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing and selects Drawing View1.
// 2. Inserts a jagged cut break.
// 3. Examine the drawing and Immediate window.
//
// NOTE: Because this drawing document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            View swView = default(View);
            BreakLine swBreakLine = default(BreakLine);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawingDoc = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Activate and select the view to which to add jagged cut break
            status = swDrawingDoc.ActivateView("Drawing View1");
            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelectionManager.GetSelectedObject6(1, -1);
            swBreakLine = (BreakLine)swView.InsertBreak3(2, -0.0291950859897372, 0.0198236302285804, (int)swBreakLineStyle_e.swBreakLine_Jagged, 2, true);
            Debug.Print("Break line style (5 = jagged cut): " + swBreakLine.Style);
            Debug.Print(" Shape intensity: " + swBreakLine.ShapeIntensity);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Jagged Cut Break Example (VBA)

This example shows how to insert a jagged cut break in a drawing view.

```
'-----
' Preconditions:
' 1. Verify that the drawing to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing and selects Drawing View1.
' 2. Inserts a jagged cut break.
' 3. Examine the drawing and Immediate window.
'
' NOTE: Because this drawing document is used elsewhere,
' do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim swBreakLine As SldWorks.BreakLine
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawingDoc = swModel
    Set swModelDocExt = swModel.Extension

    ' Activate and select the view to which to add jagged cut break
    status = swDrawingDoc.ActivateView("Drawing View1")
    status = swModelDocExt.SelectByID("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionManager = swModel.SelectionManager
    Set swView = swSelectionManager.GetSelectedObject6(1, -1)
    Set swBreakLine = swView.InsertBreak3(2, -2.91950859897372E-02, 1.98236302285804E-02, swBreakLineStyle_e.swBreakLine_Jagged, 2, True)
    Debug.Print "Break line style (5 = jagged cut): " & swBreakLine.Style
    Debug.Print " Shape intensity: " & swBreakLine.ShapeIntensity

End Sub
```

Insert Jagged Cut Break Example (VB.NET)

This example shows how to insert a jagged cut break in a drawing view.

```

-----
Preconditions:
1. Verify that the drawing to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified drawing and selects Drawing View1.
2. Inserts a jagged cut break.
3. Examine the drawing and Immediate window.

NOTE: Because this drawing document is used elsewhere,
do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDrawingDoc As DrawingDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionManager As SelectionMgr
        Dim swView As View
        Dim swBreakLine As BreakLine
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\box.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawingDoc = swModel
        swModelDocExt = swModel.Extension

        ' Activate and select the view to which to add jagged cut break
        status = swDrawingDoc.ActivateView("Drawing View1")
        status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionManager = swModel.SelectionManager
        swView = swSelectionManager.GetSelectedObject6(1, -1)
        swBreakLine = swView.InsertBreak3(2, -0.0291950859897372, 0.0198236302285804, swBreakLineStyle_e.swBreakLine_Jagged, 2, True)
        Debug.Print("Break line style (5 = jagged cut): " & swBreakLine.Style)
        Debug.Print(" Shape intensity: " & swBreakLine.ShapeIntensity)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Insert Join Feature Example (C#)

This example shows how to insert a join feature.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open exists.
// 2. Verify that c:\temp exists.
// 3. Verify that c:\temp\Part1\arm2.sldprt does not exist. If it does,
// delete or drag it to a different folder.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Edits the assembly, selects a face, and inserts a new part.
// 3. Selects the components for the join feature and inserts a
// join feature, which is a feature of Part1^arm2<1> in the
// the FeatureManager design tree.
// 4. Accesses the join feature
// 5. Gets the number of joined components.
// 6. Iterates through the components, prints the name of
// each component, and, if an Inplace mate, prints
// its mate component names and Inplace mate types.
// 7. Examine the Immediate window, FeatureManager design tree, and
// the graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace JoinFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Face2 swFace = default(Face2);
            SketchManager swSketchManager = default(SketchManager);
            Feature swFeature = default(Feature);
            JoinFeatureData swJoinFeatureData = default(JoinFeatureData);
            object swSelObj = null;
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int info = 0;
            int state = 0;
            Component2 swComponent = null;
            object singleComponent = null;
            object[] components = null;
            object[] mates = null;
            object singleMate = null;
            Mate2 swMate = null;
            MateInPlace swMateInPlace = null;
            int numMateEntities = 0;
            int typeOfMate = 0;
            int i = 0;

            //Open assembly document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\arm2.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);
            swAssemblyDoc = (AssemblyDoc)swModel;

            //Edit the assembly, select a face, and insert a new part
            swAssemblyDoc.EditPart2(true, false, ref info);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", 0.0102799485791252, 0.00285108269579837, -0.00454660000001184, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelectionMgr.GetSelectedObject6(1, -1);
            swSelObj = (object)swFace;
            state = swAssemblyDoc.InsertNewPart2("C:\\temp\\Part1^arm2.sldprt", swSelObj);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swModel.EditUndo2(1);

            //Select the components for the join feature and insert a join feature
            status = swModelDocExt.SelectByID2("SecondGrip-1@arm2", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("pincap-1@arm2", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("pincap-4@arm2", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("pincap-5@arm2", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("pincap-6@arm2", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swAssemblyDoc.InsertJoin2(true, false);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Join1@Part1^arm2-1@arm2", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);

            //Access the join feature, which is a feature of Part1^arm2<1> in the
            //the FeatureManager design tree
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swJoinFeatureData = (JoinFeatureData)swFeature.GetDefinition();
            swJoinFeatureData.AccessSelections(swModel, null);

            //Get the number of joined components
            Debug.Print("Number of joined components: " + swJoinFeatureData.GetJoinedPartsCount());

            swFeature.ReleaseSelectionAccess();

            Debug.Print("");

            //Get components
```

```

components = (object[])swAssemblyDoc.GetComponents(false);
foreach (Object singleComponent in components)
{
    swComponent = (Component2)singleComponent;
    //Print name of component
    Debug.Print("Name of component: " + swComponent.Name2);
    //Get mates
    mates = (Object[])swComponent.GetMates();
    if ((mates != null))
    {
        foreach (Object singleMate in mates)
        {
            //Get mate type
            if (singleMate is Mate2)
            {
                swMate = (Mate2)singleMate;
                typeOfMate = swMate.Type;
            }
            //If Inplace mate, print mate component name and Inplace mate type
            if (singleMate is MateInPlace)
            {
                swMateInPlace = (MateInPlace)singleMate;
                numMateEntities = swMateInPlace.GetMateEntityCount();
                for (i = 0; i <= numMateEntities - 1; i++)
                {
                    Debug.Print(" Mate component name: " + swMateInPlace.get_MateComponentName(i));
                    Debug.Print(" Type of Inplace mate: " + swMateInPlace.get_MateEntityType(i));
                }
            }
        }
    }
    Debug.Print("");
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Insert Join Feature Example (VBA)

This example shows how to insert a join feature.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Verify that c:\temp exists.
' 3. Verify that c:\temp\Part1^arm2.sldprt does not exist. If it does,
'    delete or drag it to a different folder.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Edits the assembly, selects a face, and inserts a new part.
' 3. Selects the components for the join feature and inserts a
'    join feature, which is a feature of Part1^arm2<1> in the
'    the FeatureManager design tree.
' 4. Accesses the join feature
' 5. Gets the number of joined components.
' 6. Iterates through the components, prints the name of
'    each component, and if an Inplace mate,
'    prints its mate component names and Inplace mate types.
' 7. Examine the Immediate window, FeatureManager design tree, and
'    the graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swAssemblyDoc As SldWorks.AssemblyDoc
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFace As SldWorks.Face2
Dim swSketchManager As SldWorks.SketchManager
Dim swFeature As SldWorks.Feature
Dim swJoinFeatureData As SldWorks.JoinFeatureData
Dim swSelObj As Object
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim info As Long
Dim state As Long
Dim swComponent As SldWorks.Component2
Dim singleComponent As Variant
Dim components As Variant
Dim mates As Variant
Dim singleMate As Variant
Dim swMate As SldWorks.Mate2
Dim swMateInPlace As SldWorks.MateInPlace
Dim numMateEntities As Long
Dim typeOfMate As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\arm2.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssemblyDoc = swModel

    'Edit the assembly, select a face, and insert a new part
    swAssemblyDoc.EditPart2 True, False, info
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", 1.02799485791252E-02, 2.85108269579837E-03, -4.54660000001184E-03, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFace = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swSelObj = swFace
    state = swAssemblyDoc.InsertNewPart2("C:\temp\Part1^arm2.sldprt", swSelObj)
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    swModel.EditUndo2 1

    'Select the components for the join feature and insert a join feature
    status = swModelDocExt.SelectByID2("secondGrip-1@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("pincap-1@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("pincap-4@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("pincap-5@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("pincap-6@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swAssemblyDoc.InsertJoin2(True, False)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Join1@Part1^arm2-1@arm2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

    'Access the join feature, which is a feature of Part1^arm2<1> in the
    'the FeatureManager design tree
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swJoinFeatureData = swFeature.GetDefinition
    swJoinFeatureData.AccessSelections swModel, Nothing
```

```

'Get the number of joined components
Debug.Print "Number of joined components: " & swJoinFeatureData.GetJoinedPartsCount

swJoinFeatureData.ReleaseSelectionAccess

Debug.Print ""

'Get components
components = swAssemblyDoc.GetComponents(False)
For Each singleComponent In components
    Set swComponent = singleComponent
    'Print name of component
    Debug.Print "Name of component: " & swComponent.Name2
    'Get mates
    mates = swComponent.GetMates()
    If (Not IsEmpty(mates)) Then
        For Each singleMate In mates
            'Get mate type
            If TypeOf singleMate Is SldWorks.Mate2 Then
                Set swMate = singleMate
                typeOfMate = swMate.Type
            End If
            'If Inplace mate, print mate component name and Inplace mate type
            If TypeOf singleMate Is SldWorks.MateInPlace Then
                Set swMateInPlace = singleMate
                numMateEntities = swMateInPlace.GetMateEntityCount
                For i = 0 To numMateEntities - 1
                    Debug.Print "    Mate component name: " & swMateInPlace.MateComponentName(i)
                    Debug.Print "    Type of Inplace mate: " & swMateInPlace.MateEntityType(i)
                Next i
            End If
        Next
    End If
    Debug.Print ""
Next
End Sub

```

Insert Join Feature Example (VB.NET)

This example shows how to insert a join feature.

```

-----
Preconditions:
1. Verify that the specified assembly document to open exists.
2. Verify that c:\temp exists.
3. Verify that c:\temp\Part1\arm2.sldprt does not exist. If it does,
   delete or drag it to a different folder.
4. Open the Immediate window.

Postconditions:
1. Opens the specified assembly document.
2. Edits the assembly, selects a face, and inserts a new part.
3. Selects the components for the join feature and inserts a
   join feature, which is a feature of Part1\arm2<1> in the
   the FeatureManager design tree.
4. Accesses the join feature
5. Gets the number of joined components.
6. Iterates through the components, prints the name of
   each component, and, if an Inplace mate, prints
   its mate component names and Inplace mate types.
7. Examine the Immediate window, FeatureManager design tree, and
   the graphics area.

NOTE: Because the assembly is used elsewhere, do not save changes.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swAssemblyDoc As AssemblyDoc
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFace As Face2
        Dim swSketchManager As SketchManager
        Dim swFeature As Feature
        Dim swJoinFeatureData As JoinFeatureData
        Dim swSelObj As Object
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim info As Integer
        Dim state As Integer
        Dim swComponent As Component2
        Dim singleComponent As Object
        Dim components() As Object
        Dim mates() As Object
        Dim singleMate As Object
        Dim swMate As Mate2
        Dim swMateInPlace As MateInPlace
        Dim numMateEntities As Integer
        Dim typeOfMate As Integer
        Dim i As Integer

        'Open assembly document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\arm2.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swAssemblyDoc = swModel

        'Edit the assembly, select a face, and insert a new part
        swAssemblyDoc.EditPart2(True, False, info)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("", "FACE", 0.0102799485791252, 0.00285108269579837, -0.00454660000001184, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFace = swSelectionMgr.GetSelectedObject6(1, -1)
        swSelObj = swFace
        state = swAssemblyDoc.InsertNewPart2("C:\temp\Part1\arm2.sldprt", swSelObj)
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swModel.EditUndo2(1)

        'Select the components for the join feature and insert a join feature
        status = swModelDocExt.SelectByID2("secondGrip-1@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("pincap-1@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("pincap-4@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("pincap-5@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("pincap-6@arm2", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swAssemblyDoc.InsertJoin2(True, False)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Join1@Part1\arm2-1@arm2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

        'Access the join feature, which is a feature of Part1\arm2<1> in the
        'the FeatureManager design tree
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swJoinFeatureData = swFeature.GetDefinition
        swJoinFeatureData.AccessSelections(swModel, Nothing)

        'Get the number of joined components
        Debug.Print("Number of joined components: " & swJoinFeatureData.GetJoinedPartsCount)

        swFeature.ReleaseSelectionAccess()

        Debug.Print("")

        'Get components
    
```

```

components = swAssemblyDoc.GetComponents(False)
For Each singleComponent In components
    swComponent = singleComponent
    'Print name of component
    Debug.Print("Name of component: " & swComponent.Name2)
    'Get mates
    mates = swComponent.GetMates()
    If Not mates Is Nothing Then
        For Each singleMate In mates
            'Get mate type
            If TypeOf singleMate Is Mate2 Then
                swMate = singleMate
                typeOfMate = swMate.Type
            End If
            'If Inplace mate, print mate component name and Inplace mate type
            If TypeOf singleMate Is MateInPlace Then
                swMateInPlace = singleMate
                numMateEntities = swMateInPlace.GetMateEntityCount
                For i = 0 To numMateEntities - 1
                    Debug.Print(" Mate component name: " & swMateInPlace.MateComponentName(i))
                    Debug.Print(" Type of Inplace mate: " & swMateInPlace.MateEntityType(i))
                Next i
            End If
        Next
    End If
    Debug.Print("")
Next

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Insert Linear and Circular Note Patterns (C#)

This example shows how to insert linear and circular note patterns in a drawing.

```
//-----
//
// Preconditions: Verify that the specified drawing document
// to open exists.
//
// Postconditions:
// 1. Inserts a note in the drawing and selects the note.
// 2. Inserts a linear note pattern (2 instances, including
// the original note) in the drawing.
// 3. Inserts a circular note pattern (4 instances, including
// the original note) in the drawing.
//
// NOTE: Because the model is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace InsertLinearCircularNotePatternsCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            Note swNote = default(Note);
            bool status = false;
            int errors = 0;
            int warnings = 0;

            // Open drawing document, activate sheet, and make it the active document
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.sldrw", 3, 0, "", ref errors, ref warnings);
            swApp.ActivateDoc2("foodprocessor - Sheet1", false, ref errors);
            swDrawingDoc = (DrawingDoc)swApp.ActiveDoc;

            // Insert a note
            swNote = (Note)swModel.InsertNote("Test inserting linear and circular note patterns");

            // Select the just-inserted note
            status = swModel.Extension.SelectByID2("DetailItem174@Sheet1", "NOTE", 0.2558797881203, 0.3700526, 0, false, 0, null, 0);

            // Create a linear note pattern using the selected note
            status = swDrawingDoc.InsertLinearNotePattern(2, 1, 0.01, 0.01, 0.7853981633975, 1.570796326795, "");

            // Create a circular pattern using the selected note
            status = swDrawingDoc.InsertCircularNotePattern(0.075, 4.03202193189, -4, 1.570796326795, true, "");

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Insert Linear and Circular Note Patterns (VBA)

This example shows how to insert linear and circular note patterns in a drawing.

```
'-----  
' Preconditions: Verify that the specified drawing document  
' to open exists.  
'  
' Postconditions:  
' 1. Inserts a note in the drawing and selects the note.  
' 2. Inserts a linear note pattern (2 instances, including the  
'    original note) in the drawing.  
' 3. Inserts a circular note pattern (4 instances, including  
'    the original note) in the drawing.  
'  
' NOTE: Because the model is used elsewhere, do not save  
' changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawingDoc As SldWorks.DrawingDoc  
Dim swNote As SldWorks.Note  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
  
Set swApp = Application.SldWorks  
  
' Open drawing document, activate sheet, and make it the active document  
Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw", 3, 0,  
"", errors, warnings)  
swApp.ActivateDoc2 "foodprocessor - Sheet1", False, errors  
Set swDrawingDoc = swApp.ActiveDoc  
  
' Insert a note  
Set swNote = swModel.InsertNote("Test inserting linear and circular note patterns")  
  
' Select the just-inserted note  
status = swModel.Extension.SelectByID("DetailItem174@Sheet1", "NOTE", 0.2558797881203, 0.3700526, 0, False, 0, Nothing, 0)  
  
' Create a linear note pattern using the selected note  
status = swDrawingDoc.InsertLinearNotePattern(2, 1, 0.01, 0.01, 0.7853981633975, 1.570796326795, "")  
  
' Create a circular pattern using the selected note  
status = swDrawingDoc.InsertCircularNotePattern(0.075, 4.03202193189, -4, 1.570796326795, True, "")  
  
End Sub
```

Insert Linear and Circular Note Patterns (VB.NET)

This example shows how to insert linear and circular note patterns in a drawing.

```

'-----
' Preconditions: Verify that the specified drawing document
' to open exists.
'
' Postconditions:
' 1. Inserts a note in the drawing and selects the note.
' 2. Inserts a linear note pattern (2 instances, including the
'    original note) in the drawing.
' 3. Inserts a circular note pattern (4 instances, including
'    the original note) in the drawing.
'
' NOTE: Because the model is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDrawingDoc As DrawingDoc
        Dim swNote As Note
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        ' Open drawing document, activate sheet, and make it the active document
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", 3, 0, "", errors, warnings)
        swApp.ActivateDoc2("foodprocessor - Sheet1", False, errors)
        swDrawingDoc = swApp.ActiveDoc

        ' Insert a note
        swNote = swModel.InsertNote("Test inserting linear and circular note patterns")

        ' Select the just-inserted note
        status = swModel.Extension.SelectByID2("DetailItem174@Sheet1", "NOTE", 0.2558797881203, 0.3700526, 0, False, 0, Nothing, 0)

        ' Create a linear note pattern using the selected note
        status = swDrawingDoc.InsertLinearNotePattern(2, 1, 0.01, 0.01, 0.7853981633975, 1.570796326795, "")

        ' Create a circular pattern using the selected note
        status = swDrawingDoc.InsertCircularNotePattern(0.075, 4.03202193189, -4, 1.570796326795, True, "")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert Lofted Bend Feature Example (C#)

This example shows how to insert a lofted bend feature in a sheet metal part and get the lofted bend feature data.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part.
// 2. Creates a sketch on Front Plane, a reference plane parallel to
// Front Plane, and a sketch on that reference plane.
// 3. Selects the sketches and inserts a lofted bend.
// 4. Inspect the Immediate window, FeatureManager design, and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature feat;
        RefPlane refPlane;
        SketchSegment skSegment;
        LoftedBendsFeatureData lbfd;

        bool boolstatus;

        public void Main()
        {
            // Open new part and create a sketch, a reference plane, and another sketch
            // on that reference plane
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
            skSegment = (SketchSegment)Part.SketchManager.CreateLine(0.0, 0.0, 0.0, 0.024074, 0.024074, 0.0);
            skSegment = (SketchSegment)Part.SketchManager.CreateLine(0.024074, 0.024074, 0.0, 0.076952, 0.024074, 0.0);
            skSegment = (SketchSegment)Part.SketchManager.CreateLine(0.076952, 0.024074, 0.0, 0.101026, 0.0, 0.0);
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            refPlane = (RefPlane)Part.FeatureManager.InsertRefPlane(0, 0.05, 0, 0, 0, 0);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.SketchManager.InsertSketch(true);
            skSegment = (SketchSegment)Part.SketchManager.CreateLine(-0.031383, 0.0, 0.0, 0.047146, 0.060616, 0.0);
            skSegment = (SketchSegment)Part.SketchManager.CreateLine(0.047146, 0.060616, 0.0, 0.058036, 0.060616, 0.0);
            skSegment = (SketchSegment)Part.SketchManager.CreateLine(0.058036, 0.060616, 0.0, 0.129686, 0.002436, 0.0);
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);

            // Select the sketches for the lofted bend feature
            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 1, null, 0);
            boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, true, 1, null, 0);

            // Insert a lofted bend feature with two bends
            feat = Part.FeatureManager.InsertSheetMetalLoftedBend2(0, 0.0007366, false, 0.0007366, true, (int)swLoftedBendFacetOptions_e.swBendsPerTransit
ionSegment, 0, 2, 0, 0);

            // Get lofted bend feature data
            lbfd = (LoftedBendsFeatureData)feat.GetDefinition();
            Debug.Print("Number of sketch profiles in this feature: " + lbfd.GetProfileCount());
            Debug.Print("Thickness: " + lbfd.Thickness);
            Debug.Print("Reverse thickness direction? " + lbfd.Direction);
            Debug.Print("Faceting option as defined in swLoftedBendFacetOptions_e: " + lbfd.FacetingOption);
            Debug.Print("Faceting option value: " + lbfd.FacetValue);
            Debug.Print("Formed? " + lbfd.FormedMethod);
            Debug.Print("Calculate facet transitions using vertexes? " + lbfd.ReferToEndPoint);
        }
    }
}

public SldWorks swApp;
```

Insert Lofted Bend Feature Example (VBA)

This example shows how to insert a lofted bend feature in a sheet metal part and get the lofted bend feature data.

```

'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part.
' 2. Creates a sketch on Front Plane, a reference plane parallel to
'    Front Plane, and a sketch on that reference plane.
' 3. Selects the sketches and inserts a lofted bend.
' 4. Inspect the Immediate window, FeatureManager design, and graphics area.
'-----
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim refPlane As SldWorks.refPlane
Dim skSegment As SldWorks.SketchSegment
Dim feat As SldWorks.Feature
Dim lbfd As SldWorks.LoftedBendsFeatureData
Dim boolstatus As Boolean

Option Explicit

Sub main()

    Set swApp = Application.SldWorks

    ' Open new part and create a sketch, a reference plane, and another sketch
    ' on that reference plane
    Set Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    Set skSegment = Part.SketchManager.CreateLine(0#, 0#, 0#, 0.024074, 0.024074, 0#)
    Set skSegment = Part.SketchManager.CreateLine(0.024074, 0.024074, 0#, 0.076952, 0.024074, 0#)
    Set skSegment = Part.SketchManager.CreateLine(0.076952, 0.024074, 0#, 0.101026, 0#, 0#)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set refPlane = Part.FeatureManager.InsertRefPlane(8, 0.05, 0, 0, 0)
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.SketchManager.InsertSketch True
    Set skSegment = Part.SketchManager.CreateLine(-0.031383, 0#, 0#, 0.047146, 0.060616, 0#)
    Set skSegment = Part.SketchManager.CreateLine(0.047146, 0.060616, 0#, 0.058036, 0.060616, 0#)
    Set skSegment = Part.SketchManager.CreateLine(0.058036, 0.060616, 0#, 0.129686, 0.002436, 0#)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True

    ' Select the sketches for the lofted bend feature
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 1, Nothing, 0)

    ' Insert a lofted bend feature with two bends
    Set feat = Part.FeatureManager.InsertSheetMetalLoftedBend2(0, 0.0007366, False, 0.0007366, True, swBendsPerTransitionSegment, 0, 2, 0, 0)

    ' Get lofted bend feature data
    Set lbfd = feat.GetDefinition
    Debug.Print "Number of sketch profiles in this feature: " & lbfd.GetProfileCount
    Debug.Print "Thickness: " & lbfd.Thickness
    Debug.Print "Reverse thickness direction? " & lbfd.Direction
    Debug.Print "Faceting option as defined in swLoftedBendFacetOptions_e: " & lbfd.FacetingOption
    Debug.Print "Faceting option value: " & lbfd.FacetValue
    Debug.Print "Formed? " & lbfd.FormedMethod
    Debug.Print "Calculate facet transitions using vertexes? " & lbfd.ReferToEndPoint

End Sub

```

Insert Lofted Bend Feature Example (VB.NET)

This example shows how to insert a lofted bend feature in a sheet metal part and get the lofted bend feature data.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Open the Immediate window.
Postconditions:
1. Opens a new part.
2. Creates a sketch on Front Plane, a reference plane parallel to
Front Plane, and a sketch on that reference plane.
3. Selects the sketches and inserts a lofted bend.
4. Inspect the Immediate window, FeatureManager design, and graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim refPlane As RefPlane
    Dim skSegment As SketchSegment
    Dim feat As Feature
    Dim lbfd As LoftedBendsFeatureData
    Dim boolstatus As Boolean

    Sub main()

        ' Open new part and create a sketch, a reference plane, and another sketch
        ' on that reference plane
        Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        boolstatus = Part.Extension.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        Part.ClearSelection2(True)
        Part.SketchManager.InsertSketch(True)
        skSegment = Part.SketchManager.CreateLine(0.0#, 0.0#, 0.0#, 0.024074, 0.024074, 0.0#)
        skSegment = Part.SketchManager.CreateLine(0.024074, 0.024074, 0.0#, 0.076952, 0.024074, 0.0#)
        skSegment = Part.SketchManager.CreateLine(0.076952, 0.024074, 0.0#, 0.101026, 0.0#, 0.0#)
        Part.ClearSelection2(True)
        Part.SketchManager.InsertSketch(True)
        boolstatus = Part.Extension.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        refPlane = Part.FeatureManager.InsertRefPlane(8, 0.05, 0, 0, 0, 0)
        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        Part.SketchManager.InsertSketch(True)
        skSegment = Part.SketchManager.CreateLine(-0.031383, 0.0#, 0.0#, 0.047146, 0.060616, 0.0#)
        skSegment = Part.SketchManager.CreateLine(0.047146, 0.060616, 0.0#, 0.058036, 0.060616, 0.0#)
        skSegment = Part.SketchManager.CreateLine(0.058036, 0.060616, 0.0#, 0.129686, 0.002436, 0.0#)
        Part.ClearSelection2(True)
        Part.SketchManager.InsertSketch(True)

        ' Select the sketches for the lofted bend feature
        boolstatus = Part.Extension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
        boolstatus = Part.Extension.SelectByID("Sketch2", "SKETCH", 0, 0, 0, True, 1, Nothing, 0)

        ' Insert a lofted bend feature with two bends
        feat = Part.FeatureManager.InsertSheetMetalLoftedBend2(0, 0.0007366, False, 0.0007366, True, swLoftedBendFacetOptions_e.swBendsPerTransitionSegmen
t, 0, 2, 0, 0)

        ' Get lofted bend feature data
        lbfd = feat.GetDefinition
        Debug.Print("Number of sketch profiles in this feature: " & lbfd.GetProfileCount)
        Debug.Print("Thickness: " & lbfd.Thickness)
        Debug.Print("Reverse thickness direction? " & lbfd.Direction)
        Debug.Print("Faceting option as defined in swLoftedBendFacetOptions_e: " & lbfd.FacetingOption)
        Debug.Print("Faceting option value: " & lbfd.FacetValue)
        Debug.Print("Formed? " & lbfd.FormedMethod)
        Debug.Print("Calculate facet transitions using vertexes? " & lbfd.ReferToEndPoint)

    End Sub

    Public swApp As SldWorks

End Class

```

Insert Macro Feature Example (C++ COM)

This example shows how to insert a macro feature.

```
//-----
void APITestFunction()
{
    HRESULT          hr = S_OK;
    CComPtr<ISldWorks>    pSldWorks;
    CComPtr<IModelDoc2>    pModel;
    CComQIPtr<IPartDoc>    pPart;
    CComPtr<IBody2>        pBody;
    CComPtr<IFeatureManager>    pFeatMgr;
    CComPtr<IFeature>        pFeat;
    BSTR              sMacroMethods[9];
    long              i = -1;
    VARIANT_BOOL      bRet = VARIANT_FALSE;
    // Connect to SOLIDWORKS Design
    pSldWorks = TheApplication->GetSWApp();
    ASSERT(pSldWorks);
    hr = pSldWorks->get_IActiveDoc2(&pModel);
    ASSERT(pModel);
    pPart = pModel;
    ASSERT(pPart);
    hr = pPart->IBodyObject2(&pBody);
    ASSERT(pBody);
    hr = pModel->get_FeatureManager(&pFeatMgr);
    ASSERT(pFeatMgr);
    sMacroMethods[0] = _T("");
    sMacroMethods[1] = _T("");
    sMacroMethods[2] = _T("");
    sMacroMethods[3] = _T("");
    sMacroMethods[4] = _T("");
    sMacroMethods[5] = _T("");
    sMacroMethods[6] = _T("");
    sMacroMethods[7] = _T("");
    sMacroMethods[8] = _T("");
    hr = pFeatMgr->InsertMacroFeature(
        CComBSTR(_T("CeTestMacro")),
        CComBSTR(_T("Sample.SimpleExtrude")),
        sMacroMethods,
        0,
        NULL,
        NULL,
        NULL,
        pBody,
        swMacroFeatureSecurityByDefault,
        &pFeat);
    ASSERT(pFeat);
} //void APITestFunction()
// -----
```

Insert Magnetic Line Example (C#)

This example shows how to insert a magnetic line in a drawing sheet.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\Assem20.slddrw.
// 2. Open the Immediate window.
//
// Postconditions: At each System.Diagnostics.Debugger.Break() statement,
// observe the Immediate window and drawing, then press F5:
// 1. Inserts a magnetic line.
// 2. Attaches two notes to the magnetic line.
// 3. Detaches one note from the magnetic line.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertMagneticLine_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DrawingDoc swDraw;
        Note swNote;
        Annotation anno;
        MathUtility MathUtil;
        MathPoint StartPoint;
        MathPoint EndPoint;
        MathPoint NewLocation;
        MagneticLine MagLine;
        object startPointArray;
        double[] startPointCoords = new double[3];
        object endPointArray;
        double[] endPointCoords = new double[3];
        object vAnnPos;
        Sheet myDrawingSheet;
        bool boolstatus;

        SelectionMgr swSelMgr;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)Part;
            swSelMgr = (SelectionMgr)Part.SelectionManager;

            boolstatus = Part.Extension.SelectByID("DetailItem350@Drawing View1", "NOTE", 0.1999417828571, 0.1868875085714, 0,
false, 0, null, 0);
            swNote = (Note)swSelMgr.GetSelectedObject6(1, 0);

            MathUtil = (MathUtility)swApp.GetMathUtility();

            startPointCoords[0] = 0.00792759673091;
            startPointCoords[1] = 0.0114535920683;
            startPointCoords[2] = 0;
            startPointArray = startPointCoords;
            StartPoint = (MathPoint)MathUtil.CreatePoint((startPointArray));

            endPointCoords[0] = 0.2681801120698;
            endPointCoords[1] = 0.2077950154354;
            endPointCoords[2] = 0;
            endPointArray = endPointCoords;
            EndPoint = (MathPoint)MathUtil.CreatePoint((endPointArray));

            myDrawingSheet = (Sheet)swDraw.GetCurrentSheet();
            System.Diagnostics.Debugger.Break();

            MagLine = myDrawingSheet.InsertMagneticLine(StartPoint, EndPoint);
            MagLine.EqualSpacing = true;
            Debug.Print("Number of magnetic lines on this sheet: " + myDrawingSheet.GetMagneticLinesCount());
            System.Diagnostics.Debugger.Break();

            Debug.Print("First note attached to magnetic line? " + MagLine.AddNote(swNote, 1E-06));
            // If EqualSpacing is True then Position is ignored
            Debug.Print("Number of notes attached to the magnetic line: " + MagLine.GetNotesCount());
            System.Diagnostics.Debugger.Break();

            Debug.Print("Angle in radians of the magnetic line: " + MagLine.Angle);
            Debug.Print("Length in meters of the magnetic line: " + MagLine.Length);
        }
    }
}
```

```

        boolstatus = Part.Extension.SelectByID2("DetailItem352@Drawing View3", "NOTE", 0.1944023085714, 0.1194905714286, 0,
false, 0, null, 0);
        swNote = (Note)swSelMgr.GetSelectedObject6(1, 0);
        Debug.Print("Second note attached to magnetic line? " + MagLine.AddNote(swNote, 1E-06));
        // This fails because note belongs to a different view
        anno = (Annotation)swNote.GetAnnotation();
        Debug.Print("Number of notes attached to the magnetic line: " + MagLine.GetNotesCount());
        System.Diagnostics.Debugger.Break();

        boolstatus = Part.Extension.SelectByID2("DetailItem351@Drawing View1", "NOTE", 0.1944023085714, 0.1194905714286, 0,
false, 0, null, 0);
        swNote = (Note)swSelMgr.GetSelectedObject6(1, 0);
        anno = (Annotation)swNote.GetAnnotation();
        vAnnPos = anno.GetPosition();
        Debug.Print("Second note attached to magnetic line? " + MagLine.AddNote(swNote, 1E-06));
        // This succeeds because note belongs to same view as previously added note
        Debug.Print("Number of notes attached to the magnetic line: " + MagLine.GetNotesCount());
        System.Diagnostics.Debugger.Break();

        // Detach a note from the magnetic line
        NewLocation = (MathPoint)MathUtil.CreatePoint((vAnnPos));
        Debug.Print("Note detached from magnetic line? " + MagLine.RemoveNote(swNote, NewLocation));
        // Move the note to its original position
        Debug.Print("Number of notes attached to the magnetic line: " + MagLine.GetNotesCount());

    }

    public SldWorks swApp;

}

}

```

Insert Magnetic Line Example (VBA)

This example shows how to insert a magnetic line in a drawing sheet.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\Assem20.slddrw.
' 2. Open the Immediate window.
'
' Postconditions: At each Stop statement, observe the Immediate window
' and drawing, then press F5:
' 1. Inserts a magnetic line.
' 2. Attaches two notes to the magnetic line.
' 3. Detaches one note from the magnetic line.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim swDraw As SldWorks.DrawingDoc
Dim swNote As SldWorks.Note
Dim anno As SldWorks.Annotation
Dim MathUtil As SldWorks.MathUtility
Dim startPoint As SldWorks.MathPoint
Dim endPoint As SldWorks.MathPoint
Dim NewLocation As SldWorks.MathPoint
Dim MagLine As SldWorks.MagneticLine
Dim startPointArray As Variant
Dim startPointCoords() As Double
Dim endPointArray As Variant
Dim endPointCoords() As Double
Dim myDrawingSheet As SldWorks.Sheet
Dim vAnnPos As Variant
Dim boolstatus As Boolean
Dim swSelMgr As SldWorks.SelectionMgr

Option Explicit
Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    Set swDraw = Part
    Set swSelMgr = Part.SelectionManager

    boolstatus = Part.Extension.SelectByID2("DetailItem350@Drawing View1", "NOTE", 0.1999417828571, 0.1868875085714, 0, False, 0,
    Nothing, 0)
    Set swNote = swSelMgr.GetSelectedObject6(1, 0)

    Set MathUtil = swApp.GetMathUtility()

    ReDim startPointCoords(0 To 2) As Double
    startPointCoords(0) = 0.00792759673091
    startPointCoords(1) = 0.01114535920683
    startPointCoords(2) = 0
    startPointArray = startPointCoords
    Set startPoint = MathUtil.CreatePoint((startPointArray))

    ReDim endPointCoords(0 To 2) As Double
    endPointCoords(0) = 0.2681801120698
    endPointCoords(1) = 0.2077950154354
    endPointCoords(2) = 0
    endPointArray = endPointCoords
    Set endPoint = MathUtil.CreatePoint((endPointArray))

    Set myDrawingSheet = Part.GetCurrentSheet()
    Stop

    Set MagLine = myDrawingSheet.InsertMagneticLine(startPoint, endPoint)
    MagLine.EqualSpacing = True
    Debug.Print "Number of magnetic lines on this sheet: " & myDrawingSheet.GetMagneticLinesCount
    Stop

    Debug.Print "First note attached to magnetic line? " & MagLine.AddNote(swNote, 0.000001) ' If EqualSpacing is True then
    Position is ignored
    Debug.Print "Number of notes attached to the magnetic line: " & MagLine.GetNotesCount
    Stop

    Debug.Print "Angle in radians of the magnetic line: " & MagLine.Angle
    Debug.Print "Length in meters of the magnetic line: " & MagLine.Length

    boolstatus = Part.Extension.SelectByID2("DetailItem352@Drawing View3", "NOTE", 0.1944023085714, 0.1194905714286, 0, False, 0,
    Nothing, 0)
    Set swNote = swSelMgr.GetSelectedObject6(1, 0)

```

```

    Debug.Print "Second note attached to magnetic line? " & MagLine.AddNote(swNote, 0.000001) ' This fails because note belongs to
a different view
    Set anno = swNote.GetAnnotation
    Debug.Print "Number of notes attached to the magnetic line: " & MagLine.GetNotesCount
    Stop

    boolstatus = Part.Extension.SelectByID("DetailItem351@Drawing View1", "NOTE", 0.1944023085714, 0.1194905714286, 0, False, 0,
Nothing, 0)
    Set swNote = swSelMgr.GetSelectedObject6(1, 0)
    Set anno = swNote.GetAnnotation
    vAnnPos = anno.GetPosition
    Debug.Print "Second note attached to magnetic line? " & MagLine.AddNote(swNote, 0.000001) ' This succeeds because note belongs
to same view as previously added note
    Debug.Print "Number of notes attached to the magnetic line: " & MagLine.GetNotesCount
    Stop

    ' Detach a note from the magnetic line
    Set NewLocation = MathUtil.CreatePoint((vAnnPos))
    Debug.Print "Note detached from magnetic line? " & MagLine.RemoveNote(swNote, NewLocation) ' Move the note to its original
position
    Debug.Print "Number of notes attached to the magnetic line: " & MagLine.GetNotesCount
End Sub

```

Insert Magnetic Line Example (VB.NET)

This example shows how to insert a magnetic line in a drawing sheet.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\Assem20.slddrw.
' 2. Open the Immediate window.
'
' Postconditions: At each Stop statement, observe the Immediate window
' and drawing, then press F5:
' 1. Inserts a magnetic line.
' 2. Attaches two notes to the magnetic line.
' 3. Detaches one note from the magnetic line.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim swDraw As DrawingDoc
    Dim swNote As Note
    Dim anno As Annotation
    Dim MathUtil As MathUtility
    Dim StartPoint As MathPoint
    Dim EndPoint As MathPoint
    Dim NewLocation As MathPoint
    Dim MagLine As MagneticLine
    Dim startPointArray As Object
    Dim startPointCoords(2) As Double
    Dim endPointArray As Object
    Dim endPointCoords(2) As Double
    Dim vAnnPos As Object
    Dim myDrawingSheet As Sheet
    Dim boolstatus As Boolean
    Dim swSelMgr As SelectionMgr

    Sub main()

        Part = swApp.ActiveDoc
        swDraw = Part
        swSelMgr = Part.SelectionManager

        boolstatus = Part.Extension.SelectByID2("DetailItem350@Drawing View1", "NOTE", 0.1999417828571, 0.1868875085714, 0, False,
0, Nothing, 0)
        swNote = swSelMgr.GetSelectedObject6(1, 0)

        MathUtil = swApp.GetMathUtility()

        startPointCoords(0) = 0.00792759673091
        startPointCoords(1) = 0.01114535920683
        startPointCoords(2) = 0
        startPointArray = startPointCoords
        StartPoint = MathUtil.CreatePoint((startPointArray))

        endPointCoords(0) = 0.2681801120698
        endPointCoords(1) = 0.2077950154354
        endPointCoords(2) = 0
        endPointArray = endPointCoords
        EndPoint = MathUtil.CreatePoint((endPointArray))

        myDrawingSheet = Part.GetCurrentSheet()
        Stop

        MagLine = myDrawingSheet.InsertMagneticLine(StartPoint, EndPoint)
        MagLine.EqualSpacing = True
        Debug.Print("Number of magnetic lines on this sheet: " & myDrawingSheet.GetMagneticLinesCount)
        Stop

        Debug.Print("First note attached to magnetic line? " & MagLine.AddNote(swNote, 0.000001)) ' If EqualSpacing is True then
Position is ignored
        Debug.Print("Number of notes attached to the magnetic line: " & MagLine.GetNotesCount)
        Stop

        Debug.Print("Angle in radians of the magnetic line: " & MagLine.Angle)
        Debug.Print("Length in meters of the magnetic line: " & MagLine.Length)

        boolstatus = Part.Extension.SelectByID2("DetailItem352@Drawing View3", "NOTE", 0.1944023085714, 0.1194905714286, 0, False,
0, Nothing, 0)
        swNote = swSelMgr.GetSelectedObject6(1, 0)
        Debug.Print("Second note attached to magnetic line? " & MagLine.AddNote(swNote, 0.000001)) ' This fails because note
belongs to a different view
        anno = swNote.GetAnnotation
    
```

```

        Debug.Print("Number of notes attached to the magnetic line: " & MagLine.GetNotesCount)
        Stop

        boolstatus = Part.Extension.SelectByID2("DetailItem351@Drawing View1", "NOTE", 0.1944023085714, 0.1194905714286, 0, False,
0, Nothing, 0)
        swNote = swSelMgr.GetSelectedObject6(1, 0)
        anno = swNote.GetAnnotation
        vAnnPos = anno.GetPosition
        Debug.Print("Second note attached to magnetic line? " & MagLine.AddNote(swNote, 0.000001)) ' This succeeds because note
belongs to same view as previously added note
        Debug.Print("Number of notes attached to the magnetic line: " & MagLine.GetNotesCount)
        Stop

        ' Detach a note from the magnetic line
        NewLocation = MathUtil.CreatePoint((vAnnPos))
        Debug.Print("Note detached from magnetic line? " & MagLine.RemoveNote(swNote, NewLocation)) ' Move the note to its
original position
        Debug.Print("Number of notes attached to the magnetic line: " & MagLine.GetNotesCount)

    End Sub

    Public swApp As SldWorks

End Class

```

Insert Mate Load Reference Example (C#)

This example shows how to insert a mate load reference.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Gets the mate where to add a load reference.
// 3. Selects a supplemental face for the load reference.
// 4. Inserts a mate load reference.
// 5. Examine the Immediate window.
//
// NOTE: Because the assembly document is used elsewhere, do
// not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace MateLoadReferenceCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Mate2 swMate = default(Mate2);
            MateLoadReference swMateLoadRef = default(MateLoadReference);
            Feature swFeat = default(Feature);
            Component2 swComponent = default(Component2);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool status = false;

            //Open the assembly document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\wrench.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);

            swAssemblyDoc = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get the mate where to add the load reference
            status = swModelDocExt.SelectByID2("Concentric4", "MATE", 0, 0, 0, false, 0, null, 0);
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, 0);
            swMate = (Mate2)swFeat.GetSpecificFeature2();

            //Insert the load reference using the selected mate and supplemental face
            status = swModelDocExt.SelectByID2("", "FACE", 0.087090587167495, -0.00524931403344908, 0.00483048001655106, true, 0, null, 0);
            swMateLoadRef = (MateLoadReference)swAssemblyDoc.InsertLoadReference(swMate);
            Debug.Print("Name of load reference added to " + swFeat.Name + " mate = " + swMateLoadRef.Name);
            Debug.Print("Number of supplemental faces of the mate load reference for Component1 = " + swMateLoadRef.GetFacesCount(0));
            swComponent = (Component2)swMateLoadRef.get_Component(0);
            Debug.Print("Name of Component1 = " + swComponent.Name2);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Mate Load Reference Example (VBA)

This example shows how to insert a mate load reference.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Gets the mate where to add a load reference.
' 3. Selects a supplemental face for the load reference.
' 4. Inserts a mate load reference.
' 5. Examine the Immediate window.
'
' NOTE: Because the assembly document is used elsewhere, do
' not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssemblyDoc As SldWorks.AssemblyDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swMate As SldWorks.Mate2
Dim swMateLoadRef As SldWorks.MateLoadReference
Dim swFeat As SldWorks.Feature
Dim swComponent As SldWorks.Component2
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks

    'Open the assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wrench.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssemblyDoc = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    'Get the mate where to add the load reference
    status = swModelDocExt.SelectByID2("Concentric4", "MATE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)
    Set swMate = swFeat.GetSpecificFeature2

    'Insert the load reference using the selected mate and supplemental face
    status = swModelDocExt.SelectByID2("", "FACE", 0.087090587167495, -5.24931403344908E-03, 4.83048001655106E-03, True, 0, Nothing, 0)
    Set swMateLoadRef = swAssemblyDoc.InsertLoadReference(swMate)

    Debug.Print "Name of load reference added to " & swFeat.Name & " mate = " & swMateLoadRef.Name
    Debug.Print "Number of supplemental faces of the mate load reference for Component1 = " & swMateLoadRef.GetFacesCount(0)
    Set swComponent = swMateLoadRef.Component(0)
    Debug.Print "Name of Component1 = " & swComponent.Name2

End Sub
```

Insert Mate Load Reference Example (VB.NET)

This example shows how to insert a mate load reference.

```

-----
Preconditions:
1. Verify that the specified assembly document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified assembly document.
2. Gets the mate where to add a load reference.
3. Selects a supplemental face for the load reference.
4. Inserts a mate load reference.
5. Examine the Immediate window.
NOTE: Because the assembly document is used elsewhere, do
not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssemblyDoc As AssemblyDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swMate As Mate2
        Dim swMateLoadRef As MateLoadReference
        Dim swFeat As Feature
        Dim swComponent As Component2
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim status As Boolean

        'Open the assembly document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wrench.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swAssemblyDoc = swModel
        swSelMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension

        'Get the mate where to add the load reference
        status = swModelDocExt.SelectByID2("Concentric4", "MATE", 0, 0, 0, False, 0, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, 0)
        swMate = swFeat.GetSpecificFeature2

        'Insert the load reference using the selected mate and supplemental face
        status = swModelDocExt.SelectByID2("", "FACE", 0.087090587167495, -0.00524931403344908, 0.00483048001655106, True, 0, Nothing, 0)
        swMateLoadRef = swAssemblyDoc.InsertLoadReference(swMate)
        Debug.Print("Name of load reference added to " & swFeat.Name & " mate = " & swMateLoadRef.Name)
        Debug.Print("Number of supplemental faces of the mate load reference for Component1 = " & swMateLoadRef.GetFacesCount(0))
        swComponent = swMateLoadRef.Component(0)
        Debug.Print("Name of Component1 = " & swComponent.Name2)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert MidSurface in Component Example (C#)

This example shows how to insert a midsurface feature in a component.

```
//-----
// Preconditions:
// 1. Open an assembly that contains at least one component
// that contains a solid body.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a midsurface feature in the component.
// 2. Gets the number of faces in the midsurface feature.
// 3. Examine the Immediate window.
// 4. Expand the component in the FeatureManager design tree
// in which the midsurface feature was inserted to
// verify step 1.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swExt;
        SelectionMgr swSelMgr;
        Component2 swComp;
        AssemblyDoc swAssem;
        FeatureManager featMgr;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            featMgr = (FeatureManager)swModel.FeatureManager;
            swAssem = (AssemblyDoc)swModel;
            object[] vComponents = null;
            vComponents = (object[])swAssem.GetComponents(true);
            swComp = (Component2)vComponents[0];
            object[] vBodies = null;
            vBodies = (object[])swComp.GetBodies2((int)swBodyType_e.swSolidBody);
            if ((vBodies != null))
            {
                Body2 pBody = default(Body2);
                pBody = (Body2)vBodies[0];
                MidSurface3 midSurf = default(MidSurface3);
                swModel = (ModelDoc2)swComp.GetModelDoc2();
                Debug.Print("Component in which to insert midsurface feature: " + swModel.GetPathName());
                midSurf = (MidSurface3)featMgr.InsertMidSurface(pBody, swModel, 0.5, true);
                Debug.Print("Face count: " + midSurf.GetFaceCount());
            }
            else
            {
                Debug.Print("Open a different assembly in which the specified body is a solid body.");
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert MidSurface in Component Example (VBA)

This example shows how to insert a midsurface feature in a component.

```
'-----
' Preconditions:
' 1. Open an assembly that contains at least one component
'    that contains a solid body.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a midsurface feature in the component.
' 2. Gets the number of faces in the midsurface feature.
' 3. Examine the Immediate window.
' 4. Expand the component in the FeatureManager design tree
'    in which the midsurface feature was inserted to
'    verify step 1.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swComp As SldWorks.Component2
Dim swAssem As SldWorks.AssemblyDoc
Dim featMgr As SldWorks.FeatureManager

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    Set featMgr = swModel.FeatureManager
    Set swAssem = swModel

    Dim vComponents As Variant
    vComponents = swAssem.GetComponents(True)
    Set swComp = vComponents(0)
    Dim vBodies As Variant
    vBodies = swComp.GetBodies2(swBodyType_e.swSolidBody)
    If Not IsEmpty(vBodies) Then
        Dim pBody As Body2
        Set pBody = vBodies(0)
        Dim midSurf As MidSurface3
        Set swModel = swComp.GetModelDoc2
        Debug.Print "Component in which to insert midsurface feature: " & swModel.GetPathName
        Set midSurf = featMgr.InsertMidSurface(pBody, swModel, 0.5, True)
        Debug.Print "Face count: " & midSurf.GetFaceCount
    Else
        Debug.Print "Open a different assembly in which the specified body is a solid body."
    End If

End Sub
```

Insert MidSurface in Component Example (VB.NET)

This example shows how to insert a midsurface feature in a component.

```

'-----
' Preconditions:
' 1. Open an assembly that contains at least one component
'    that contains a solid body.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a midsurface feature in the component.
' 2. Gets the number of faces in the midsurface feature.
' 3. Examine the Immediate window.
' 4. Expand the component in the FeatureManager design tree
'    in which the midsurface feature was inserted to
'    verify step 1.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim swComp As Component2
    Dim swAssem As AssemblyDoc
    Dim featMgr As FeatureManager

    Public Sub main()

        swModel = swApp.ActiveDoc
        swExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        featMgr = swModel.FeatureManager
        swAssem = swModel

        Dim vComponents As Object
        vComponents = swAssem.GetComponents(True)
        swComp = vComponents(0)
        Dim vBodies As Object
        vBodies = swComp.GetBodies2(swBodyType_e.swSolidBody)
        If Not IsNothing(vBodies) Then
            Dim pBody As Body2
            pBody = vBodies(0)
            Dim midSurf As MidSurface3
            swModel = swComp.GetModelDoc2
            Debug.Print("Component in which to insert midsurface feature: " & swModel.GetPathName)
            midSurf = featMgr.InsertMidSurface(pBody, swModel, 0.5, True)
            Debug.Print("Face count: " & midSurf.GetFaceCount)
        Else
            Debug.Print("Open a different assembly in which the specified body is a solid body.")
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Insert Model Annotations Example (C#)

This example shows how to automatically insert a model's dimensions marked for drawings into a drawing.

```
//-----
// Preconditions:
// 1. Assembly document to open exists.
// 2. Run the macro.
//
// Postconditions:
// 1. A new drawing document is opened.
// 2. A drawing view of the assembly document is created.
// 3. The dimensions in the assembly document that are marked for drawings,
//    including any duplicate dimensions, appear in the drawing view.
// 4. The dimensions in the drawing, which are annotations,
//    are selected and marked.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace SelectAnnotationsCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            DrawingDoc swDrawing;
            SelectionMgr swSelMgr;
            View swView;
            object[] annotations;
            object selAnnot;
            Annotation swAnnotation;
            SelectData swSelData;
            int mark;
            string retval;
            bool status;

            retval = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplateDrawing);
            swModel = (ModelDoc2)swApp.NewDocument(retval, 0, 0, 0);
            swDrawing = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Create drawing from assembly
            swView = (View)swDrawing.CreateDrawViewFromModelView3("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2024\\samples\\tutorial\\api\\wrench.sldasm", "**Front", 0.1314541543147, 0.1407887187817, 0);

            // Select and activate the view
            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            status = swDrawing.ActivateView("Drawing View1");

            swModel.ClearSelection2(true);

            // Insert the annotations marked for the drawing
            annotations = (object[])swDrawing.InsertModelAnnotations4((int)swImportModelItemsSource_e.swImportModelItemsFromEntireModel, (int)swInsertAnnotation_e.swInsertDimensionsMark
edForDrawing, true, false, false, false, false, false);

            // Select and mark each annotation
            swSelData = swSelMgr.CreateSelectData();
            mark = 0;

            foreach (object annot in annotations)
            {
                selAnnot = annot;
                swAnnotation = (Annotation)selAnnot;
                status = swAnnotation.Select3(true, swSelData);
                swSelData.Mark = mark;
                mark = mark + 1;
            }

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```


Insert Model Annotations Example (VBA)

This example shows how to automatically insert a model's dimensions marked for drawings into a drawing.

```
'-----
' Preconditions:
' 1. Assembly document to open exists.
' 2. Run the macro.
'
' Postconditions:
' 1. A new drawing document is opened.
' 2. A drawing view of the assembly document is created.
' 3. The dimensions in the assembly document that are marked for drawings,
'    including any duplicate dimensions, appear in the drawing view.
' 4. The dimensions in the drawing, which are annotations,
'    are selected and marked.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim swSelmgr As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim annotations As Variant
Dim annot As Variant
Dim swAnnotation As SldWorks.Annotation
Dim swSelData As SldWorks.SelectData
Dim mark As Long
Dim retval As String
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks

    retval = swApp.GetUserPreferenceStringValue(swDefaultTemplateDrawing)
    Set swModel = swApp.NewDocument(retval, 0, 0, 0)
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension
    Set swSelmgr = swModel.SelectionManager

    ' Create drawing from assembly
    Set swView = swDrawing.CreateDrawViewFromModelView3("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\api\wrench.sldasm",
    "**Front", 0.1314541543147, 0.1407887187817, 0)

    ' Select and activate the view
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    status = swDrawing.ActivateView("Drawing View1")

    swModel.ClearSelection2 True

    ' Insert the annotations marked for the drawing
    annotations = swDrawing.InsertModelAnnotations4(0, swInsertDimensionsMarkedForDrawing, True, False, False, False, False, False)

    ' Select and mark each annotation
    Set swSelData = swSelmgr.CreateSelectData
    mark = 0

    For Each annot In annotations
        Set swAnnotation = annot
        status = swAnnotation.Select3(True, swSelData)
        swSelData.mark = mark
        mark = mark + 1
    Next

End Sub
```

Insert Model Annotations Example (VB.NET)

This example shows how to automatically insert a model's dimensions marked for drawings into a drawing.

```
'
'-----
' Preconditions:
' 1. Assembly document to open exists.
' 2. Run the macro.
'
' Postconditions:
' 1. A new drawing document is opened.
' 2. A drawing view of the assembly document is created.
' 3. The dimensions in the assembly document that are marked for drawings,
'    including any duplicate dimensions, appear in the drawing view.
' 4. The dimensions in the drawing, which are annotations,
'    are selected and marked.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
Dim swModel As ModelDoc2
Dim swModelDocExt As ModelDocExtension
Dim swDrawing As DrawingDoc
Dim swSelmgr As SelectionMgr
Dim swView As View
Dim annotations As Object
Dim annot As Object
Dim swAnnotation As Annotation
Dim swSelData As SelectData
Dim mark As Integer
Dim retval As String
Dim status As Boolean
```

Sub main()

```
    retval = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplateDrawing)
    swModel = swApp.NewDocument(retval, 0, 0, 0)
    swDrawing = swModel
    swModelDocExt = swModel.Extension
    swSelmgr = swModel.SelectionManager

    ' Create drawing from assembly
    swView = swDrawing.CreateDrawViewFromModelView3("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2024\samples\tutorial\api\wrench.sldasm", "*Front", 0.1314541543147, 0.1407887187817, 0)

    ' Select and activate the view
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    status = swDrawing.ActivateView("Drawing View1")

    swModel.ClearSelection2(True)

    ' Insert the annotations marked for the drawing
    annotations = swDrawing.InsertModelAnnotations4(swImportModelItemsSource_e.swImportModelItemsFromEntireModel, swInsertAnnotation_e.swInsertDimensionsMarkedForDrawing, True,
False, False, False, False, False)

    ' Select and mark each annotation
    swSelData = swSelmgr.CreateSelectData
    mark = 0

    For Each annot In annotations
        swAnnotation = annot
        status = swAnnotation.Select3(True, swSelData)
        swSelData.Mark = mark
        mark = mark + 1
    Next
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Insert New Instance of Virtual Component (C#)

This example shows how to:

- create an assembly document.
- insert a new part as a virtual component in the assembly document.
- insert a new instance of the virtual component in the assembly document.

```
//-----
// Preconditions: None
//
// Postconditions:
// 1. An assembly document is created.
// 2. A virtual part is inserted in the assembly document.
// 3. A new instance of the virtual part is inserted
//    in the assembly document.
// 4. Examine the FeatureManager design tree to
//    verify steps 2 and 3.
// 5. Close the assembly document without saving the modified
//    documents.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace AddNewVirtualPartsCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            string asmTemplate = null;
            asmTemplate = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplateAssembly);

            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.NewDocument(asmTemplate, 0, 0, 0);

            SelectionMgr swSelMgr = default(SelectionMgr);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            if (swModel.Extension.SelectByID("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0) == false)
            {
                Debug.Print("Failed to select Front plane; check feature name.");
                return;
            }

            Feature swPlaneFeature = default(Feature);
            swPlaneFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            RefPlane swPlane = default(RefPlane);
            swPlane = (RefPlane)swPlaneFeature.GetSpecificFeature2();

            AssemblyDoc swAssem = default(AssemblyDoc);
            swAssem = (AssemblyDoc)swModel;

            int lResult = 0;
            Component2 swVirtComp = default(Component2);
            lResult = swAssem.InsertNewVirtualPart(swPlane, out swVirtComp);

            if (lResult == (int)swInsertNewPartErrorCode_e.swInsertNewPartError_NoError)
            {
                Component2 swSecondComp = default(Component2);
                swSecondComp = (Component2)swAssem.AddComponent5(swVirtComp.GetPathName(),
                    (int)swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", false, "", 0.1, 0, 0);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Insert New Instance of Virtual Component (VBA)

This example shows how to:

- create an assembly document.
- insert a new part as a virtual component in the assembly document.
- insert a new instance of the virtual component in the assembly document.

```
'-----
' Preconditions: None
'
' Postconditions:
' 1. An assembly document is created.
' 2. A virtual component is inserted in the assembly document.
' 3. A new instance of the virtual component is inserted
'    in the assembly document.
' 4. Examine the FeatureManager design tree to
'    verify steps 2 and 3.
' 5. Close the assembly document without saving the modified
'    documents.
'-----
```

Option Explicit

```
Sub main()
    Dim swApp As SldWorks.SldWorks
    Set swApp = Application.SldWorks

    Dim asmTemplate As String
    asmTemplate = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplateAssembly)

    Dim swModel As SldWorks.ModelDoc2
    Set swModel = swApp.NewDocument(asmTemplate, 0, 0, 0)

    Dim swSelMgr As SldWorks.SelectionMgr
    Set swSelMgr = swModel.SelectionManager

    If swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0) = False Then
        Debug.Print "Failed to select Front plane; check feature name."
        Exit Sub
    End If

    Dim swPlaneFeature As SldWorks.Feature
    Set swPlaneFeature = swSelMgr.GetSelectedObject6(1, -1)
    Dim swPlane As SldWorks.refPlane
    Set swPlane = swPlaneFeature.GetSpecificFeature2

    Dim swAssem As SldWorks.AssemblyDoc
    Set swAssem = swModel

    Dim lResult As Long
    Dim swVirtComp As SldWorks.Component2
    lResult = swAssem.InsertNewVirtualPart(swPlane, swVirtComp)

    If lResult = swInsertNewPartError_NoError Then
        Dim swSecondComp As SldWorks.Component2
        Set swSecondComp = swAssem.AddComponent5(swVirtComp.GetPathName, swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", 0.1, 0, 0)
    End If

End Sub
```

Insert New Instance of Virtual Component (VB.NET)

This example shows how to:

- create an assembly document.
- insert a new part as a virtual component in the assembly document.
- insert a new instance of the virtual component in the assembly document.

```

'-----
' Preconditions: None
'-----
' Postconditions:
' 1. An assembly document is created.
' 2. A virtual part is inserted in the assembly document.
' 3. A new instance of the virtual part is inserted
'    in the assembly document.
' 4. Examine the FeatureManager design tree to
'    verify steps 2 and 3.
' 5. Close the assembly document without saving the modified
'    documents.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim asmTemplate As String
        asmTemplate = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplateAssembly)

        Dim swModel As ModelDoc2
        swModel = swApp.NewDocument(asmTemplate, 0, 0, 0)

        Dim swSelMgr As SelectionMgr
        swSelMgr = swModel.SelectionManager

        If swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0) = False Then
            Debug.Print("Failed to select Front plane; check feature name.")
            Exit Sub
        End If

        Dim swPlaneFeature As Feature
        swPlaneFeature = swSelMgr.GetSelectedObject6(1, -1)
        Dim swPlane As RefPlane
        swPlane = swPlaneFeature.GetSpecificFeature2

        Dim swAssem As AssemblyDoc
        swAssem = swModel

        Dim lResult As Integer
        Dim swVirtComp As Component2
        swVirtComp = Nothing
        lResult = swAssem.InsertNewVirtualPart(swPlane, swVirtComp)

        If lResult = swInsertNewPartErrorCode_e.swInsertNewPartError_NoError Then
            Dim swSecondComp As Component2
            swSecondComp = swAssem.AddComponent5(swVirtComp.GetPathName,
            swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", 0.1, 0, 0)
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert New Virtual Assembly Example (C#)

This example shows how to insert an assembly as a virtual component into the main assembly or selected sub-assembly.

```
//-----  
// Preconditions: Open an assembly document.  
//  
// Postconditions: A new virtual component displays in the  
// FeatureManager design tree.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace InsertNewVirtualAssembly_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 swDoc;  
        AssemblyDoc swADoc;  
        Component2 swComp;  
        long status;  
        public void Main()  
        {  
            swDoc = (ModelDoc2)swApp.ActiveDoc;  
            swADoc = (AssemblyDoc)swDoc;  
            swComp = null;  
            status = swADoc.InsertNewVirtualAssembly(out swComp);  
            if ((swComp == null))  
            {  
                System.Windows.Forms.MessageBox.Show("Virtual component did not get created.");  
            }  
            else  
            {  
                Debug.Print("New virtual component: " + swComp.Name2);  
                Debug.Print("Is virtual: " + swComp.IsVirtual);  
            }  
        }  
        public SldWorks swApp;  
    }  
}
```

Insert New Virtual Assembly Example (VBA)

This example shows how to insert an assembly as a virtual component into the main assembly or selected sub-assembly.

.....
' Preconditions: Open an assembly document.

'
' Postconditions: A new virtual component displays in the
' FeatureManager design tree.
.....

```
Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim swADoc As SldWorks.AssemblyDoc
Dim swComp As SldWorks.Component2
Dim status As Long

Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set swDoc = swApp.ActiveDoc
    Set swADoc = swDoc
    Set swComp = Nothing
    status = swADoc.InsertNewVirtualAssembly(swComp)
    If (swComp Is Nothing) Then
        MsgBox ("Virtual component did not get created.")
    Else
        Debug.Print ("New virtual component: " & swComp.Name2)
        Debug.Print ("Is virtual: " & swComp.IsVirtual)
    End If
End Sub
```

Insert New Virtual Assembly Example (VB.NET)

This example shows how to insert an assembly as a virtual component into the main assembly or selected sub-assembly.

.....
' Preconditions: Open an assembly document.

'
' Postconditions: A new virtual component displays in the
' FeatureManager design tree.
.....

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Imports System.Diagnostics

Partial Class SolidWorksMacro

Dim swDoc As ModelDoc2

Dim swADoc As AssemblyDoc

Dim swComp As Component2

Dim status As Long

Sub main()

swDoc = swApp.ActiveDoc

swADoc = swDoc

swComp = Nothing

status = swADoc.InsertNewVirtualAssembly(swComp)

If (swComp Is Nothing) Then

MsgBox("Virtual component did not get created.")

Else

Debug.Print("New virtual component: " & swComp.Name2)

Debug.Print("Is virtual: " & swComp.IsVirtual)

End If

End Sub

Public swApp As SldWorks

End Class

Insert New Virtual Component Example (C#)

This example shows how to insert a new part as a virtual component in an assembly and save it to an external file.

```
//-----
// Preconditions:
// 1. Add a reference to Microsoft Scripting Runtime (right-click
// the name of the project in the Project Explorer and click Add Reference >
// the Browse tab > C:\windows\system32\scriun.dll > OK.
// 2. Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.
// 3. Select a planar face on the assembly.
// 4. Open the Immediate window.
// 5. Step through this macro by pressing F8.
//
// Postconditions:
// 1. Inserts a new part as a virtual component in the assembly.
// 2. Attempts to save the virtual component to an external file.
// 3. Examine the Immediate window and FeatureManager design tree.
//
// NOTE: Because this assembly is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using Scripting;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        AssemblyDoc swAssy;
        Component2 swComponent;
        SelectionMgr swSelMgr;
        FileSystemObject objFSO;
        string compName;
        object[] splits;
        long status;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;

            // Get the pre-selected planar face
            Face2 swFeature = default(Face2);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Face2)swSelMgr.GetSelectedObject6(1, 0);

            // Create the part and insert it as a virtual component
            // in the assembly
            status = swAssy.InsertNewVirtualPart(swFeature, out swComponent);

            if (status == 1)
            {
                Debug.Print("Virtual component inserted!");
                Debug.Print("Name of virtual component: " + swComponent.Name2);

                // Check to see if the part is a virtual component
                Debug.Print("Is component virtual? " + swComponent.IsVirtual);

                objFSO = new Scripting.FileSystemObject();

                splits = swComponent.Name2.Split('^');
                compName = objFSO.GetParentFolderName(swModel.GetPathName()) + "\\\" + splits[0];

                ModelDoc2 compModel = default(ModelDoc2);
                compModel = (ModelDoc2)swComponent.GetModelDoc();

                if (compModel.GetType() == (int)swDocumentTypes_e.swDocPART)
                {
                    compName = compName + ".sldprt";
                }
                else
                {
                    compName = compName + ".sldasm";
                }

                Debug.Print("Path and name of virtual component: " + compName);

                bool ret;
                ret = swComponent.SaveVirtualComponent(compName);
                if (ret)
                {
                    Debug.Print("    Virtual component saved!");
                }
                else
                {
                    Debug.Print("    Virtual component not saved!");
                    Debug.Print("    Check the folder's attributes where to save the virtual component and check your permissions to this folder.");
                }
            }
            else
            {
                Debug.Print("Error code returned when attempting to insert new part as virtual component: " + status);
            }

            swModel.ClearSelection2(true);
        }

        public SldWorks swApp;
    }
}
```

Insert New Virtual Component Example (VBA)

This example shows how to insert a new part as a virtual component in an assembly and save it to an external file.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.
' 2. Select a planar face on the assembly.
' 3. Open the Immediate window.
' 4. Step through this macro by pressing F8.
'
' Postconditions:
' 1. Inserts a new part as a virtual component in the assembly.
' 2. Attempts to save the virtual component to an external file.
' 3. Examine the Immediate window and FeatureManager design tree.
'
' NOTE: Because this assembly is used elsewhere, do not
' save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swComponent As SldWorks.Component2
Dim swSelMgr As SldWorks.SelectionMgr
Dim status As Long

Sub Main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel

    ' Get the pre-selected planar face
    Dim swFeature As SldWorks.Face2
    Set swSelMgr = swModel.SelectionManager
    Set swFeature = swSelMgr.GetSelectedObject6(1, 0)

    ' Create the part and insert it as a virtual component
    ' in the assembly
    status = swAssy.InsertNewVirtualPart(swFeature, swComponent)

    If status = 1 Then

        Debug.Print "Virtual component inserted!"
        Debug.Print "Name of virtual component: " & swComponent.Name2

        ' Check to see if the part is a virtual component
        Debug.Print " Is component virtual? " & swComponent.IsVirtual

        Dim objFSO As Object
        Dim objFile As Object
        Dim compName As String
        Dim splits As Variant

        Set objFSO = CreateObject("Scripting.FileSystemObject")
        Set objFile = objFSO.GetFile(swModel.GetPathName)

        splits = Split(swComponent.Name2, "^")
        compName = objFSO.GetParentFolderName(objFile) & "\" & splits(0)

        If swComponent.GetModelDoc2.GetType = swDocPART Then
            compName = compName & ".sldprt"
        Else
            compName = compName & ".sldasm"
        End If

        Debug.Print " Path and name of virtual component: " & compName

        Dim ret As Boolean
        ret = swComponent.SaveVirtualComponent(compName)
        If ret Then
            Debug.Print " Virtual component saved!"
        Else
            Debug.Print " Virtual component not saved!"
            Debug.Print " Check the folder's attributes where to save the virtual component and check your permissions to this folder."
        End If

    Else
        Debug.Print "Error code returned when attempting to insert new part as virtual component: " & status
    End If

    swModel.ClearSelection2 (True)

End Sub

```

Insert New Virtual Component Example (VB.NET)

This example shows how to insert a new part as a virtual component in an assembly and save it to an external file.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.
' 2. Select a planar face on the assembly.
' 3. Open the Immediate window.
' 4. Step through this macro by pressing F8.
'
' Postconditions:
' 1. Inserts a new part as a virtual component in the assembly.
' 2. Attempts to save the virtual component to an external file.
' 3. Examine the Immediate window and FeatureManager design tree.
'
' NOTE: Because this assembly is used elsewhere, do not
' save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swAssy As AssemblyDoc
    Dim swComponent As Component2
    Dim swSelMgr As SelectionMgr
    Dim objFSO As Object
    Dim objFile As Object
    Dim compName As String
    Dim splits As Object
    Dim status As Integer

    Sub Main()

        swModel = swApp.ActiveDoc
        swAssy = swModel

        ' Get the pre-selected planar face
        Dim swFeature As Face2
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, 0)

        ' Create the part and insert it as a virtual component
        ' in the assembly
        status = swAssy.InsertNewVirtualPart(swFeature, swComponent)

        If status = 1 Then
            Debug.Print("Virtual component inserted!")
            Debug.Print("Name of virtual component: " & swComponent.Name2)

            ' Check to see if the part is a virtual component
            Debug.Print(" Is component virtual? " & swComponent.IsVirtual)

            objFSO = CreateObject("Scripting.FileSystemObject")
            objFile = objFSO.GetFile(swModel.GetPathName)

            splits = Split(swComponent.Name2, "^")
            compName = objFSO.GetParentFolderName(objFile) & "\" & splits(0)

            Dim compModel As ModelDoc2
            compModel = swComponent.GetModelDoc

            If compModel.GetType = swDocumentTypes_e.swDocPART Then
                compName = compName & ".sldprt"
            Else
                compName = compName & ".sldasm"
            End If

            Debug.Print(" Path and name of virtual component: " & compName)

            Dim ret As Boolean
            ret = swComponent.SaveVirtualComponent(compName)
            If ret Then
                Debug.Print(" Virtual component saved!")
            Else
                Debug.Print(" Virtual component not saved!")
                Debug.Print(" Check the folder's attributes where to save the virtual component and check your permissions to this folder.")
            End If

            Else
                Debug.Print("Error code returned when attempting to insert new part as virtual component: " & status)
            End If

            swModel.ClearSelection2(True)

        End Sub

        Public swApp As SldWorks

    End Class

```

Insert Part Number Column in BOM Table Example (C#)

This example shows how to insert a part number column in a BOM table.

```
//-----
// Preconditions:
// 1. Open a drawing with a BOM table.
// 2. Right-click the BOM table, select Select,
//    and select Table.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a part number column at the end of the
//    the BOM table.
// 2. Examine the BOM table and the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace GetColumnTitleCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            bool boolstatus = false;
            SelectionMgr SelMgr = null;
            TableAnnotation theTableAnnotation = default(TableAnnotation);
            int SelObjType = 0;
            int TableAnnotationType = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            SelMgr = (SelectionMgr)swModel.SelectionManager;
            SelObjType = (int)SelMgr.GetSelectedObjectType2(1);
            if (SelObjType != (int)swSelectType_e.swSelANNOTATIONTABLES)
            {
                MessageBox.Show("Select a BOM table in the drawing before running this macro.");
                return;
            }
            theTableAnnotation = (TableAnnotation)SelMgr.GetSelectedObject6(1, -1);
            TableAnnotationType = theTableAnnotation.Type;
            if (TableAnnotationType != (int)swTableAnnotationType_e.swTableAnnotation_BillofMaterials)
            {
                MessageBox.Show("Select a BOM table in the drawing before running this example.");
                return;
            }
            DisplayTableColumnProps(theTableAnnotation);
            boolstatus = theTableAnnotation.InsertColumn((int)swTableItemInsertPosition_e.swTableItemInsertPosition_Last, 0, "New Column");
            boolstatus = theTableAnnotation.SetColumnType(theTableAnnotation.ColumnCount - 1, (int)swTableColumnTypes_e.swBomTableColumnType_PartNumber);
            DisplayTableColumnProps(theTableAnnotation);
        }

        public void DisplayTableColumnProps(TableAnnotation theTableAnnotation)
        {
            int ColCount = 0;
            int i = 0;
            int ColType = 0;
            string ColTitle = null;

            Debug.Print("Index" + "\t" + "Type" + "\t" + "Title");
            ColCount = theTableAnnotation.ColumnCount;
            for (i = 0; i <= ColCount - 1; i++)
            {
                ColType = theTableAnnotation.GetColumnType(i);
                ColTitle = theTableAnnotation.GetColumnTitle(i);
                Debug.Print("\t" + i + "\t" + ColType + "\t\t" + ColTitle);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Part Number Column in BOM Table Example (VBA)

This example shows how to insert a part number column in a BOM table.

```
'-----
' Preconditions:
' 1. Open a drawing with a BOM table.
' 2. Right-click the BOM table, select Select,
'    and select Table.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a part number column at the end of the
'    the BOM table.
' 2. Examine the BOM table and the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim SelMgr As Object
Dim theTableAnnotation As SldWorks.TableAnnotation
Dim SelObjType As Long
Dim TableAnnotationType As Long

Sub DisplayTableColumnProps(theTableAnnotation As Object)
    Dim ColCount As Long
    Dim i As Long
    Dim ColType As swTableColumnTypes_e
    Dim ColTitle As String

    Debug.Print "Index" & vbTab & "Type" & vbTab & "Title"
    ColCount = theTableAnnotation.ColumnCount

    For i = 0 To ColCount - 1
        ColType = theTableAnnotation.GetColumnType(i)
        ColTitle = theTableAnnotation.GetColumnTitle(i)
        Debug.Print i & vbTab & ColType & vbTab & ColTitle
    Next i
End Sub

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set SelMgr = swModel.SelectionManager

    SelObjType = SelMgr.GetSelectedObjectType2(1)

    If SelObjType <> swSelectType_e.swSelANNOTATIONTABLES Then
        MsgBox "Select a BOM table in the drawing before running this macro."
        End
    End If

    Set theTableAnnotation = SelMgr.GetSelectedObject6(1, -1)
    TableAnnotationType = theTableAnnotation.Type
    If TableAnnotationType <> swTableAnnotationType_e.swTableAnnotation_BillOfMaterials Then
        MsgBox "Select a BOM table in the drawing before running this example."
        End
    End If

    DisplayTableColumnProps theTableAnnotation

    boolstatus = theTableAnnotation.InsertColumn(swTableItemInsertPosition_e.swTableItemInsertPosition_Last, 0, "New Column")
    boolstatus = theTableAnnotation.SetColumnType(theTableAnnotation.ColumnCount - 1, swTableColumnTypes_e.swBomTableColumnType_PartNumber)

    DisplayTableColumnProps theTableAnnotation
End Sub
```

Insert Part Number Column in BOM Table Example (VB.NET)

This example shows how to insert a part number column in a BOM table.

```

-----
Preconditions:
1. Open a drawing with a BOM table.
2. Right-click the BOM table, select Select,
   and select Table.
3. Open the Immediate window.
Postconditions:
1. Inserts a part number column at the end of the
   the BOM table.
2. Examine the BOM table and the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim boolstatus As Boolean
        Dim SelMgr As Object
        Dim theTableAnnotation As TableAnnotation
        Dim SelObjType As Integer
        Dim TableAnnotationType As Integer

        swModel = swApp.ActiveDoc
        SelMgr = swModel.SelectionManager
        SelObjType = SelMgr.GetSelectedObjectType2(1)
        If SelObjType <> swSelectType_e.swSelANNOTATIONTABLES Then
            MsgBox("Select a BOM table in the drawing before running this macro.")
            Exit Sub
        End If
        theTableAnnotation = SelMgr.GetSelectedObject6(1, -1)
        TableAnnotationType = theTableAnnotation.Type
        If TableAnnotationType <> swTableAnnotationType_e.swTableAnnotation_BillOfMaterials Then
            MsgBox("Select a BOM table in the drawing before running this example.")
            Exit Sub
        End If
        DisplayTableColumnProps(theTableAnnotation)
        boolstatus = theTableAnnotation.InsertColumn(swTableItemInsertPosition_e.swTableItemInsertPosition_Last, 0, "New Column")
        boolstatus = theTableAnnotation.SetColumnType(theTableAnnotation.ColumnCount - 1, swTableColumnType_e.swBomTableColumnType_PartNumber)
        DisplayTableColumnProps(theTableAnnotation)

    End Sub

    Sub DisplayTableColumnProps(ByVal theTableAnnotation As Object)
        Dim ColCount As Long
        Dim i As Long
        Dim ColType As swTableColumnType_e
        Dim ColTitle As String

        Debug.Print("Index" & vbTab & "Type" & vbTab & "Title")
        ColCount = theTableAnnotation.ColumnCount
        For i = 0 To ColCount - 1
            ColType = theTableAnnotation.GetColumnType(i)
            ColTitle = theTableAnnotation.GetColumnTitle(i)
            Debug.Print(i & vbTab & ColType & vbTab & ColTitle)
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert Protusion Blend Example (C#)

This example shows how to create a loft using profiles, guide curves, and a centerline.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Creates a new part.
// 2. Creates a profile sketch.
// 3. Creates a reference plane and another profile sketch on that
//    reference plane.
// 4. Creates five curves: four guide curves and one centerline.
// 5. Selects the profile sketches, four guide curves, and the
//    centerline.
// 6. Creates a loft feature.
// 7. Examine the FeatureManager design tree and graphics area.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchSegment swSketchSegment = default(SketchSegment);
            SketchManager swSketchManager = default(SketchManager);
            RefPlane swRefPlane = default(RefPlane);
            FeatureManager swFeatureManager = default(FeatureManager);
            bool status = false;

            //Create new part
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);

            //Create profile sketch
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchManager.CreateEllipse(0, 0, 0, 0.0706113079019074, 0, 0, 0, 0.0374944141689373, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            // Create reference plane and another profile sketch
            // on that reference plane
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swRefPlane = (RefPlane)swFeatureManager.InsertRefPlane(8, 0.07, 0, 0, 0, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateEllipse(0, 0, 0, 0.0527205722070845, 0, 0, 0, 0.0154164850136235, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            // Create guide curves
            status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 0, 0, 0.0374944141689373, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Point4@Sketch2", "EXTSKETCHPOINT", 0, 0, 0.0154164850136235, 0, true, 1, null, 0);
            swModel.Insert3DSplineCurve(false);

            status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -0.0527205722070845, 0, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("Point5@Sketch1", "EXTSKETCHPOINT", -0.0706113079019074, 0, 0, true, 1, null, 0);
            swModel.Insert3DSplineCurve(false);

            status = swModelDocExt.SelectByID2("Point6@Sketch2", "EXTSKETCHPOINT", 0, -0.0154164850136235, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("Point6@Sketch1", "EXTSKETCHPOINT", 0, -0.0374944141689373, 0, true, 1, null, 0);
            swModel.Insert3DSplineCurve(false);

            status = swModelDocExt.SelectByID2("Point3@Sketch2", "EXTSKETCHPOINT", 0.0527205722070845, 0, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("Point3@Sketch1", "EXTSKETCHPOINT", 0.0706113079019074, 0, 0, true, 1, null, 0);
            swModel.Insert3DSplineCurve(false);

            // Create centerline
            status = swModelDocExt.SelectByID2("Point2@Sketch2", "EXTSKETCHPOINT", 0, 0, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("Point2@Sketch1", "EXTSKETCHPOINT", 0, 0, 0, true, 1, null, 0);
            swModel.Insert3DSplineCurve(false);

            // Create loft
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0.0706113079019074, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0.0527205722070845, 0, 0.07, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("Curve1", "REFERENCECURVES", 0.0999754519565386, 0.0447609702560072, 0.128010464418594, true, 4098, null, 0);
            status = swModelDocExt.SelectByID2("Curve2", "REFERENCECURVES", 0.037643674311596, 0.0221603475855119, 0.115437869126538, true, 8194, null, 0);
            status = swModelDocExt.SelectByID2("Curve3", "REFERENCECURVES", 0.0999909384372586, -0.000744308680111772, 0.131301605626447, true, 12290, null, 0);
            status = swModelDocExt.SelectByID2("Curve4", "REFERENCECURVES", 0.158600974878482, 0.0218780932746938, 0.129235804629445, true, 16386, null, 0);
            status = swModelDocExt.SelectByID2("Curve5", "REFERENCECURVES", 0.0998735089003162, 0.022159545044488, 0.108064927518626, true, 4, null, 0);
            swFeatureManager.InsertProtusionBlend2(false, true, false, 1, 0, 0, 1, 1, true, true, false, 0, 0, 0, true, true, true, (int)swGuideCurveInfluence_e.swGuideCurveInfluenceNextEdge);

            /// <summary>
            /// The SldWorks swApp variable is pre-assigned for you.
            /// </summary>
            public SldWorks swApp;
        }
    }
}
```

Insert Protrusion Blend Example (VBA)

This example shows how to create a loft using profiles, guide curves, and a centerline.

```

'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates a new part.
' 2. Creates a profile sketch.
' 3. Creates a reference plane and another profile sketch on that
'    reference plane.
' 4. Creates five curves: four guide curves and one centerline.
' 5. Selects the profile sketches, four guide curves, and the
'    centerline.
' 6. Creates a loft feature.
' 7. Examine the FeatureManager design tree and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSketchManager As SldWorks.SketchManager
Dim swRefPlane As SldWorks.RefPlane
Dim swFeatureManager As SldWorks.FeatureManager
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks

    'Create new part
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)

    'Create profile sketch
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swSketchManager = swModel.SketchManager
    Set swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 7.06113079019074E-02, 0, 0, 0, 3.74944141689373E-02, 0)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    ' Create reference plane and another profile sketch
    ' on that reference plane
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swRefPlane = swFeatureManager.InsertRefPlane(8, 0.07, 0, 0, 0, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 5.27205722070845E-02, 0, 0, 0, 1.54164850136235E-02, 0)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    ' Create guide curves
    status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 0, 3.74944141689373E-02, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point4@Sketch2", "EXTSKETCHPOINT", 0, 1.54164850136235E-02, 0, True, 1, Nothing, 0)
    swModel.Insert3DSplineCurve False

    status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -5.27205722070845E-02, 0, 0, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point5@Sketch1", "EXTSKETCHPOINT", -7.06113079019074E-02, 0, 0, True, 1, Nothing, 0)
    swModel.Insert3DSplineCurve False

    status = swModelDocExt.SelectByID2("Point6@Sketch2", "EXTSKETCHPOINT", 0, -1.54164850136235E-02, 0, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point6@Sketch1", "EXTSKETCHPOINT", 0, -3.74944141689373E-02, 0, True, 1, Nothing, 0)
    swModel.Insert3DSplineCurve False

    status = swModelDocExt.SelectByID2("Point3@Sketch2", "EXTSKETCHPOINT", 5.27205722070845E-02, 0, 0, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point3@Sketch1", "EXTSKETCHPOINT", 7.06113079019074E-02, 0, 0, True, 1, Nothing, 0)
    swModel.Insert3DSplineCurve False

    ' Create centerline
    status = swModelDocExt.SelectByID2("Point2@Sketch2", "EXTSKETCHPOINT", 0, 0, 0, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point2@Sketch1", "EXTSKETCHPOINT", 0, 0, 0, True, 1, Nothing, 0)
    swModel.Insert3DSplineCurve False

    ' Create loft
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 7.06113079019074E-02, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 5.27205722070845E-02, 0, 0.07, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Curve1", "REFERENCECURVES", 9.99754519565386E-02, 4.47609702560072E-02, 0.128010464418594, True, 4098, Nothing, 0)
    status = swModelDocExt.SelectByID2("Curve2", "REFERENCECURVES", 0.037643674311596, 2.21603475855119E-02, 0.115437869126538, True, 8194, Nothing, 0)
    status = swModelDocExt.SelectByID2("Curve3", "REFERENCECURVES", 9.99909384372586E-02, -7.44308680111772E-04, 0.131301605626447, True, 12290, Nothing, 0)

    status = swModelDocExt.SelectByID2("Curve4", "REFERENCECURVES", 0.158600974878482, 2.18780932746938E-02, 0.129235804629445, True, 16386, Nothing, 0)
    status = swModelDocExt.SelectByID2("Curve5", "REFERENCECURVES", 9.98735089003162E-02, 0.022159545044488, 0.108064927518626, True, 4, Nothing, 0)
    swFeatureManager.InsertProtrusionBlend2 False, True, False, 1, 0, 0, 1, 1, True, True, False, 0, 0, 0, True, True, True,
    swGuideCurveInfluence_e.swGuideCurveInfluenceNextEdge

End Sub

```

Insert Protrusion Blend Example (VB.NET)

This example shows how to create a loft using profiles, guide curves, and a centerline.

```

-----
' Preconditions: Verify that the specified part template exists.
-----
Postconditions:
1. Creates a new part.
2. Creates a profile sketch.
3. Creates a reference plane and another profile sketch on that
   reference plane.
4. Creates five curves: four guide curves and one centerline.
5. Selects the profile sketches, four guide curves, and the
   centerline.
6. Creates a loft feature.
7. Examine the FeatureManager design tree and graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchSegment As SketchSegment
        Dim swSketchManager As SketchManager
        Dim swRefPlane As RefPlane
        Dim swFeatureManager As FeatureManager
        Dim status As Boolean

        'Create new part
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prt", 0, 0, 0)

        'Create profile sketch
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchManager = swModel.SketchManager
        swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 0.0706113079019074, 0, 0, 0, 0.0374944141689373, 0)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        ' Create reference plane and another profile sketch
        ' on that reference plane
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swRefPlane = swFeatureManager.InsertRefPlane(0, 0.07, 0, 0, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 0.0527205722070845, 0, 0, 0, 0.0154164850136235, 0)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        ' Create guide curves
        status = swModelDocExt.SelectByID("Point4@Sketch1", "EXTSKETCHPOINT", 0, 0.0374944141689373, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Point4@Sketch2", "EXTSKETCHPOINT", 0, 0.0154164850136235, 0, True, 1, Nothing, 0)
        swModel.Insert3DSplineCurve(False)

        status = swModelDocExt.SelectByID("Point5@Sketch2", "EXTSKETCHPOINT", -0.0527205722070845, 0, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Point5@Sketch1", "EXTSKETCHPOINT", -0.0706113079019074, 0, 0, True, 1, Nothing, 0)
        swModel.Insert3DSplineCurve(False)

        status = swModelDocExt.SelectByID("Point6@Sketch2", "EXTSKETCHPOINT", 0, -0.0154164850136235, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Point6@Sketch1", "EXTSKETCHPOINT", -0.0374944141689373, 0, 0, True, 1, Nothing, 0)
        swModel.Insert3DSplineCurve(False)

        status = swModelDocExt.SelectByID("Point3@Sketch2", "EXTSKETCHPOINT", 0.0527205722070845, 0, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Point3@Sketch1", "EXTSKETCHPOINT", 0.0706113079019074, 0, 0, True, 1, Nothing, 0)
        swModel.Insert3DSplineCurve(False)

        ' Create centerline
        status = swModelDocExt.SelectByID("Point2@Sketch2", "EXTSKETCHPOINT", 0, 0, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Point2@Sketch1", "EXTSKETCHPOINT", 0, 0, 0, True, 1, Nothing, 0)
        swModel.Insert3DSplineCurve(False)

        ' Create loft
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0.0706113079019074, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0.0527205722070845, 0, 0.07, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Curve1", "REFERENCECURVES", 0.0999754519565386, 0.0447609702560072, 0.128010464418594, True, 4098, Nothing, 0)
        status = swModelDocExt.SelectByID("Curve2", "REFERENCECURVES", 0.037643674311596, 0.0221603475855119, 0.115437869126538, True, 8194, Nothing, 0)
        status = swModelDocExt.SelectByID("Curve3", "REFERENCECURVES", 0.0999909384372586, -0.000744308680111772, 0.131301605626447, True, 12290, Nothing, 0)

        status = swModelDocExt.SelectByID("Curve4", "REFERENCECURVES", 0.158600974878482, 0.0218780932746938, 0.129235804629445, True, 16386, Nothing, 0)
        status = swModelDocExt.SelectByID("Curve5", "REFERENCECURVES", 0.0998735089003162, 0.022159545044488, 0.108064927518626, True, 4, Nothing, 0)
        swFeatureManager.InsertProtrusionBlend2(False, True, False, 1, 0, 0, 1, 1, True, True, False, 0, 0, 0, True, True, True,
        , 0)
        swGuideCurveInfluence_e.swGuideCurveInfluenceNextEdge)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert Punch Table Example (C#)

This example shows how to insert a punch table in the flat pattern view of a sheet metal part that contains one or more forming tool features.

```
//-----
// Preconditions:
// 1. Open a drawing that contains a flat pattern view of a sheet
//    metal part with one or more forming tool features.
// 2. Modify the x- and y-coordinates in the IModelDocExtension::SelectByID2
//    methods for the specified VERTEX and FACE.
// 3. Verify that the punch table template exists.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a punch table in the drawing.
// 2. Inspect the Immediate window for punch table feature details.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace InsertPunchTable_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DrawingDoc Draw;
        bool boolstatus;
        View myView;
        PunchTableAnnotation myTableAnn;
        PunchTable MyTable;
        Feature featRet;

        object rVar;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            Draw = (DrawingDoc)Part;
            boolstatus = Draw.ActivateView("Drawing View1");
            boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);

            //Select a vertex in the drawing view to be the origin of all datums in the table
            //All X LOCATION and Y LOCATION table column values will be relative to this datum origin
            boolstatus = Part.Extension.SelectByID2("", "VERTEX", 0.1344949308616, 0.0948467785893, 0, false, 1, null, 0);

            //Select a face that contains the punches that will be annotated in the table
            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.1562337360869, 0.1059474450873, 0, true, 2, null, 0);

            myView = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObjectsDrawingView(1);

            myTableAnn = myView.InsertPunchTable(false, 0.2178779310824, 0.2022819591903, 1, "A", "C:\\Program Files\\SOLIDWORKS
Corp\\SOLIDWORKS\\lang\\english\\punchtable-standard.sldpuntbt");

            MyTable = myTableAnn.PunchTable;
            Debug.Print("Default punch table properties:");
            Debug.Print("");
            Debug.Print("Tag style (swPunchTableTagStyle_e): " + MyTable.TagStyle);
            Debug.Print("Starting tag: " + MyTable.StartingValue);
            Debug.Print("Merge cells with the punch ID for a given tag? " + MyTable.CombineSameSize);
            Debug.Print("Combine rows with the same punch ID? " + MyTable.CombineTags);
            Debug.Print("Display dual dimensions? " + MyTable.DualDimensions);
            Debug.Print("Display units for dual dimensions? " + MyTable.ShowUnits);
            Debug.Print("Punch table annotation count: " + MyTable.GetTableAnnotationCount());

            rVar = MyTable.GetTableAnnotations();

            featRet = (Feature)MyTable.GetFeature();
            Debug.Print("Feature name: " + featRet.Name);

            Part.ClearSelection2(true);
            boolstatus = Draw.ActivateSheet("Sheet1");
        }

        public SldWorks swApp;
    }
}
```

Insert Punch Table Example (VBA)

This example shows how to insert a punch table in the flat pattern view of a sheet metal part that contains one or more forming tool features.

```

'-----
' Preconditions:
' 1. Open a drawing that contains a flat pattern view of a sheet
'    metal part with one or more forming tool features.
' 2. Modify the x- and y-coordinates in the IModelDocExtension::SelectByID2
'    methods for the specified VERTEX and FACE.
' 3. Verify that the punch table template exists.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a punch table in the drawing.
' 2. Inspect the Immediate window for punch table feature details.
'-----
Dim swApp As SldWorks.SldWorks

Dim Part As SldWorks.ModelDoc2

Dim boolstatus As Boolean

Dim myView As SldWorks.View

Dim myTableAnn As SldWorks.PunchTableAnnotation

Dim MyTable As SldWorks.PunchTable

Dim featRet As SldWorks.Feature

Dim rVar As Variant

Option Explicit

Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    boolstatus = Part.ActivateView("Drawing View1")
    boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

    'Select a vertex in the drawing view to be the origin of all datums in the table
    'All X LOCATION and Y LOCATION table column values will be relative to this datum origin
    boolstatus = Part.Extension.SelectByID2("", "VERTEX", 0.1344949308616, 0.0948467785893, 0, False, 1, Nothing, 0)

    'Select a face that contains the punches that will be annotated in the table
    boolstatus = Part.Extension.SelectByID2("", "FACE", 0.1562337360869, 0.1059474450873, 0, True, 2, Nothing, 0)

    Set myView = Part.SelectionManager.GetSelectedObjectsDrawingView(1)

    Set myTableAnn = myView.InsertPunchTable(False, 0.2178779310824, 0.2022819591903, 1, "A", "C:\Program Files\SOLIDWORKS
Corp\SOLIDWORKS\lang\english\punchtable-standard.sldpuntbt")
    If myTableAnn Is Nothing Then MsgBox "Failed to insert Punch Table"

    Set MyTable = myTableAnn.PunchTable
    Debug.Print "Default punch table properties:"
    Debug.Print ""
    Debug.Print "Tag style (swPunchTableTagStyle_e): " & MyTable.TagStyle
    Debug.Print "Starting tag: " & MyTable.StartingValue
    Debug.Print "Merge cells with the punch ID for a given tag? " & MyTable.CombineSameSize
    Debug.Print "Combine rows with the same punch ID? " & MyTable.CombineTags
    Debug.Print "Display dual dimensions? " & MyTable.DualDimensions
    Debug.Print "Display units for dual dimensions? " & MyTable.ShowUnits
    Debug.Print "Punch table annotation count: " & MyTable.GetTableAnnotationCount

    rVar = MyTable.GetTableAnnotations

    Set featRet = MyTable.GetFeature
    Debug.Print "Feature name: " & featRet.Name

    Part.ClearSelection2 True
    boolstatus = Part.ActivateSheet("Sheet1")
End Sub

```

Insert Punch Table Example (VB.NET)

This example shows how to insert a punch table in the flat pattern view of a sheet metal part that contains one or more forming tool features.

```
'-----
' Preconditions:
' 1. Open a drawing that contains a flat pattern view of a sheet
'    metal part with one or more forming tool features.
' 2. Modify the x- and y-coordinates in the IModelDocExtension::SelectByID2
'    methods for the specified VERTEX and FACE.
' 3. Verify that the punch table template exists.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a punch table in the drawing.
' 2. Inspect the Immediate window for punch table feature details.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2

    Dim boolstatus As Boolean

    Dim myView As View

    Dim myTableAnn As PunchTableAnnotation

    Dim MyTable As PunchTable

    Dim featRet As Feature

    Dim rVar As Object

    Sub main()

        Part = swApp.ActiveDoc

        boolstatus = Part.ActivateView("Drawing View1")

        boolstatus = Part.Extension.SelectByID2("Drawing
View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

        'Select a vertex in the drawing
view to be the origin of all datums in the table

        'All
X LOCATION and Y LOCATION table column values will be relative to this datum
origin

        boolstatus =
Part.Extension.SelectByID2("", "VERTEX",
0.1344949308616, 0.0948467785893, 0, False,
1, Nothing,
0)

        'Select a face that contains the
punches that will be annotated in the table

        boolstatus =
Part.Extension.SelectByID2("", "FACE",
0.1562337360869, 0.1059474450873, 0, True,
2, Nothing,
0)

        myView = Part.SelectionManager.GetSelectedObjectsDrawingView(1)

        myTableAnn = myView.InsertPunchTable(False,
0.2178779310824, 0.2022819591903, 1, "A", "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\punchtable-standard.sldpuntbt")

        If myTableAnn Is Nothing Then MsgBox("Failed to insert Punch Table")

        MyTable = myTableAnn.PunchTable

        Debug.Print("Default punch table
properties:")

        Debug.Print("")
    End Sub
End Class
```

```

        Debug.Print("Tag style (swPunchTableTagStyle_e):
" & MyTable.TagStyle)

        Debug.Print("Starting tag: " & MyTable.StartingValue)

        Debug.Print("Merge cells with the
punch ID for a given tag? " & MyTable.CombineSameSize)

        Debug.Print("Combine rows with
the same punch ID? " & MyTable.CombineTags)

        Debug.Print("Display dual
dimensions? " & MyTable.DualDimensions)

        Debug.Print("Display units for
dual dimensions? " & MyTable.ShowUnits)

        Debug.Print("Punch table
annotation count: " & MyTable.GetTableAnnotationCount)

        rVar = MyTable.GetTableAnnotations

        featRet = MyTable.GetFeature

        Debug.Print("Feature name: " & featRet.Name)

        Part.ClearSelection2(True)

        boolstatus = Part.ActivateSheet("Sheet1")

End Sub

Public swApp As SldWorks

End Class

```

Insert Reference Plane Example (C#)

This example shows how to create a constraint-based angle reference plane.

```
//-----
// 1. Verify that the specified part exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a constraint-based reference plane.
// 2. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace InsertRefPlaneFeatureManagerCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            RefPlane swRefPlane = default(RefPlane);
            SelectionMgr swSelMgr = default(SelectionMgr);
            RefPlaneFeatureData swRefPlaneFeatureData = default(RefPlaneFeatureData);
            int fileerror = 0;
            int filewarning = 0;
            bool boolstatus = false;
            int planeType = 0;

            swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\plate.sldprt",
(int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref fileerror, ref filewarning);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Create a constraint-based reference plane
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.028424218552, 0.07057725774359, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.05976462601598, 0.0718389621656, 0.0001242036435087, true, 1,
null, 0);

            swRefPlane = (RefPlane)swFeatureManager.InsertRefPlane(16, 0.7853981633975, 4, 0, 0, 0);

            // Get type of the just-created reference plane
            boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swRefPlaneFeatureData = (RefPlaneFeatureData)swFeature.GetDefinition();

            planeType = swRefPlaneFeatureData.Type2;
            Debug.Print("Type of reference plane using IRefPlaneFeatureData::Type2: ");
            switch (planeType)
            {
                case 0:
                    Debug.Print(" Invalid");
                    break;
                case 1:
                    Debug.Print(" Undefined");
                    break;
            }
        }
    }
}
```

```

        case 2:
            Debug.Print(" Line Point");
            break;
        case 3:
            Debug.Print(" Three Points");
            break;
        case 4:
            Debug.Print(" Line Line");
            break;
        case 5:
            Debug.Print(" Distance");
            break;
        case 6:
            Debug.Print(" Parallel");
            break;
        case 7:
            Debug.Print(" Angle");
            break;
        case 8:
            Debug.Print(" Normal");
            break;
        case 9:
            Debug.Print(" On Surface");
            break;
        case 10:
            Debug.Print(" Standard");
            break;
        case 11:
            Debug.Print(" Constraint-based");
            break;
    }
    Debug.Print("");

    planeType = swRefPlaneFeatureData.Type;
    Debug.Print("Type of reference plane using IRefPlaneFeatureData::Type: ");
    switch (planeType)
    {
        case 0:
            Debug.Print(" Invalid");
            break;
        case 1:
            Debug.Print(" Undefined");
            break;
        case 2:
            Debug.Print(" Line Point");
            break;
        case 3:
            Debug.Print(" Three Points");
            break;
        case 4:
            Debug.Print(" Line Line");
            break;
        case 5:
            Debug.Print(" Distance");
            break;
        case 6:
            Debug.Print(" Parallel");
            break;
        case 7:
            Debug.Print(" Angle");
            break;
        case 8:

```

```

        Debug.Print(" Normal");
        break;
    case 9:
        Debug.Print(" On Surface");
        break;
    case 10:
        Debug.Print(" Standard");
        break;
    case 11:
        Debug.Print(" Constraint-based");
        break;
}
Debug.Print("");

swModel.ClearSelection2(true);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Insert Reference Plane Example (VBA)

This example shows how to create a constraint-based angle reference plane.

```
'-----
' 1. Verify that the specified part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a constraint-based reference plane.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swRefPlane As SldWorks.RefPlane
Dim swSelMgr As SldWorks.SelectionMgr
Dim swRefPlaneFeatureData As SldWorks.RefPlaneFeatureData
Dim fileerror As Long
Dim filewarning As Long
Dim boolstatus As Boolean
Dim planeType As Long

Sub main()

    Set swApp = Application.SldWorks
    swApp.OpenDoc6 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\plate.sldprt", swDocPART, swOpenDocOptions_Silent, "",
fileerror, filewarning
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager
    Set swSelMgr = swModel.SelectionManager

    ' Create a constraint-based reference plane
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.028424218552, 0.07057725774359, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("", "EDGE", 0.05976462601598, 0.0718389621656, 1.242036435087E-04, True, 1, Nothing, 0)
    Set swRefPlane = swFeatureManager.InsertRefPlane(16, 0.7853981633975, 4, 0, 0, 0)

    ' Get type of the just-created reference plane
    boolstatus = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swRefPlaneFeatureData = swFeature.GetDefinition
    planeType = swRefPlaneFeatureData.Type2
    Debug.Print "Type of reference plane using IRefPlaneFeatureData::Type2: "
    Select Case planeType
        Case 0
            Debug.Print " Invalid"
        Case 1
            Debug.Print " Undefined"
        Case 2
            Debug.Print " Line Point"
        Case 3
            Debug.Print " Three Points"
        Case 4
            Debug.Print " Line Line"
        Case 5
            Debug.Print " Distance"
        Case 6
            Debug.Print " Parallel"
        Case 7
            Debug.Print " Angle"
        Case 8
            Debug.Print " Normal"
        Case 9
            Debug.Print " On Surface"
        Case 10
            Debug.Print " Standard"
        Case 11
            Debug.Print " Constraint-based"
    End Select
    Debug.Print ""

    planeType = swRefPlaneFeatureData.Type
    Debug.Print "Type of reference plane using IRefPlaneFeatureData::Type: "
    Select Case planeType
        Case 0
            Debug.Print " Invalid"
        Case 1
            Debug.Print " Undefined"
        Case 2
            Debug.Print " Line Point"
        Case 3
            Debug.Print " Three Points"
        Case 4
            Debug.Print " Line Line"
        Case 5
            Debug.Print " Distance"
        Case 6
            Debug.Print " Parallel"
```

```
Case 7
  Debug.Print " Angle"
Case 8
  Debug.Print " Normal"
Case 9
  Debug.Print " On Surface"
Case 10
  Debug.Print " Standard"
Case 11
  Debug.Print " Constraint-based"
End Select
Debug.Print ""

swModel.ClearSelection2 True

End Sub
```

Insert Reference Plane Example (VB.NET)

This example shows how to create a constraint-based angle reference plane.

```
'-----
' 1. Verify that the specified part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a constraint-based reference plane.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swRefPlane As RefPlane
        Dim swSelMgr As SelectionMgr
        Dim swRefPlaneFeatureData As RefPlaneFeatureData
        Dim fileerror As Long
        Dim filewarning As Long
        Dim boolstatus As Boolean
        Dim planeType As Long

        swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\plate.sldprt",
swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", fileerror, filewarning)
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager
        swSelMgr = swModel.SelectionManager

        ' Create a constraint-based reference plane
        boolstatus = swModelDocExt.SelectByID("", "FACE", 0.028424218552, 0.07057725774359, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID("", "EDGE", 0.05976462601598, 0.0718389621656, 0.0001242036435087, True, 1, Nothing,
0)
        swRefPlane = swFeatureManager.InsertRefPlane(16, 0.7853981633975, 4, 0, 0, 0)

        ' Get type of the just-created reference plane
        boolstatus = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swRefPlaneFeatureData = swFeature.GetDefinition

        planeType = swRefPlaneFeatureData.Type2
        Debug.Print("Type of reference plane using IRefPlaneFeatureData::Type2: ")
        Select Case planeType
            Case 0
                Debug.Print(" Invalid")
            Case 1
                Debug.Print(" Undefined")
            Case 2
                Debug.Print(" Line Point")
            Case 3
                Debug.Print(" Three Points")
            Case 4
```

```

        Debug.Print(" Line Line")
    Case 5
        Debug.Print(" Distance")
    Case 6
        Debug.Print(" Parallel")
    Case 7
        Debug.Print(" Angle")
    Case 8
        Debug.Print(" Normal")
    Case 9
        Debug.Print(" On Surface")
    Case 10
        Debug.Print(" Standard")
    Case 11
        Debug.Print(" Constraint-based")
End Select
Debug.Print("")

```

```

planeType = swRefPlaneFeatureData.Type
Debug.Print("Type of reference plane using IRefPlaneFeatureData::Type: ")
Select Case planeType
    Case 0
        Debug.Print(" Invalid")
    Case 1
        Debug.Print(" Undefined")
    Case 2
        Debug.Print(" Line Point")
    Case 3
        Debug.Print(" Three Points")
    Case 4
        Debug.Print(" Line Line")
    Case 5
        Debug.Print(" Distance")
    Case 6
        Debug.Print(" Parallel")
    Case 7
        Debug.Print(" Angle")
    Case 8
        Debug.Print(" Normal")
    Case 9
        Debug.Print(" On Surface")
    Case 10
        Debug.Print(" Standard")
    Case 11
        Debug.Print(" Constraint-based")
End Select
Debug.Print("")

```

```

swModel.ClearSelection2(True)

```

```

End Sub

```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

```

End Class

```

Insert Reference Points Example (VBA)

This example shows how to insert six, evenly spaced referenced points on the selected edge.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Select an edge.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates six evenly spaced reference points on the
'    selected edge.
' 2. Examines the Immediate window, FeatureManager design
'    tree, and graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim vFeatArr As Variant
    Dim vFeat As Variant
    Dim swFeat As SldWorks.Feature
    Dim swRefPt As SldWorks.RefPoint
    Dim swRefPtData As SldWorks.RefPointFeatureData
    Dim swMathPt As SldWorks.MathPoint
    Dim nStatus As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Debug.Print "File = " & swModel.GetPathName

    ' An edge must be preselected
    vFeatArr = swFeatMgr.InsertReferencePoint(swRefPointAlongCurve, swRefPointAlongCurveEvenlyDistributed, 0#, 6)
    For Each vFeat In vFeatArr
        Set swFeat = vFeat
        Set swRefPt = swFeat.GetSpecificFeature2
        Set swRefPtData = swFeat.GetDefinition
        Set swMathPt = swRefPt.GetRefPoint
        Debug.Print " " & swFeat.Name
        Debug.Print " Pt = (" & swMathPt.ArrayData(0) * 1000# & ", " & swMathPt.ArrayData(1) * 1000# & ", " & swMathPt.ArrayData(2) * 1000# & ") mm"
        Debug.Print " AlongCurveOption = " & swRefPtData.AlongCurveOption
        Debug.Print " Distance = " & swRefPtData.Distance * 1000# & " mm"
        Debug.Print " Type = " & swRefPtData.Type
    Next
End Sub
```

Insert Revision Cloud into a Drawing Example (C#)

This example shows how to insert revision clouds into a drawing and access revision cloud data.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\resetsketchvisibility.slddrw.
//
// Postconditions:
// 1. Inserts an elliptical revision cloud in the drawing.
// 2. Examine the Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertRevisionCloud_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        DrawingDoc Part;
        RevisionCloud RevCloud;
        Annotation RevCloudAnno;

        bool boolstatus;

        public void Main()
        {
            Part = (DrawingDoc)swApp.ActiveDoc;
            boolstatus = Part.ActivateView("Drawing View1");

            // Create a revision cloud with an elliptical shape
            RevCloud = (RevisionCloud)Part.InsertRevisionCloud(1);
            if ((RevCloud != null))
            {
                RevCloudAnno = (Annotation)RevCloud.GetAnnotation();
                if ((RevCloudAnno != null))
                {
                    // Position the center of the elliptical revision cloud
                    boolstatus = RevCloudAnno.SetPosition(0.270847371964905, 0.553263328912467, 0);
                    RevCloud.ArcRadius = 0.00508;
                    // Create a path point on the corner of an ellipse-inscribed rectangle
                    boolstatus = RevCloud.SetPathPointAtIndex(-1, 0.378419710263212, 0.511051398694144, 0);
                    // Close the revision cloud path
                    boolstatus = RevCloud.Finalize();
                }
            }
        }

        public SldWorks swApp;
    }
}
```

Insert Revision Cloud into a Drawing Example (VBA)

This example shows how to insert revision clouds into a drawing and access revision cloud data.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\resetsketchvisibility.slddrw.  
'  
' Postconditions:  
' 1. Inserts an elliptical revision cloud in the drawing.  
' 2. Examine the graphics area.  
'  
' NOTE: Because the drawing is used elsewhere, do not save changes.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.DrawingDoc  
Dim RevCloud As SldWorks.RevisionCloud  
Dim RevCloudAnno As SldWorks.Annotation  
Dim boolstatus As Boolean  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    boolstatus = Part.ActivateView("Drawing View1")  
  
    ' Create a revision cloud with an elliptical shape  
    Set RevCloud = Part.InsertRevisionCloud(swRevisionCloudShape_Ellipse)  
    If Not RevCloud Is Nothing Then  
        Set RevCloudAnno = RevCloud.GetAnnotation()  
        If Not RevCloudAnno Is Nothing Then  
            ' Position the center of the elliptical revision cloud  
            boolstatus = RevCloudAnno.SetPosition(0.270847371964905, 0.553263328912467, 0)  
            RevCloud.ArcRadius = 0.00508  
            ' Create a path point on the corner of an ellipse-inscribed rectangle  
            boolstatus = RevCloud.SetPathPointAtIndex(-1, 0.378419710263212, 0.511051398694144, 0)  
            ' Close the revision cloud path  
            boolstatus = RevCloud.Finalize()  
        End If  
    End If  
  
End Sub
```

Insert Revision Cloud into a Drawing Example (VB.NET)

This example shows how to insert revision clouds into a drawing and access revision cloud data.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\resetsketchvisibility.slddrw.
```

```
' Postconditions:
' 1. Inserts an elliptical revision cloud in the drawing.
' 2. Examine the graphics area.
```

```
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As DrawingDoc
    Dim RevCloud As RevisionCloud
    Dim RevCloudAnno As Annotation
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
        boolstatus = Part.ActivateView("Drawing View1")
```

```
        ' Create a revision cloud with an elliptical shape
        RevCloud = Part.InsertRevisionCloud(1)
```

```
        If Not RevCloud Is Nothing Then
```

```
            RevCloudAnno = RevCloud.GetAnnotation
```

```
            If Not RevCloudAnno Is Nothing Then
```

```
                ' Position the center of the elliptical revision cloud
```

```
                boolstatus = RevCloudAnno.SetPosition(0.270847371964905, 0.553263328912467, 0)
```

```
                RevCloud.ArcRadius = 0.00508
```

```
                ' Create a path point on the corner of an ellipse-inscribed rectangle
```

```
                boolstatus = RevCloud.SetPathPointAtIndex(-1, 0.378419710263212, 0.511051398694144, 0)
```

```
                ' Close the revision cloud path
```

```
                boolstatus = RevCloud.Finalize
```

```
            End If
```

```
        End If
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Insert Scale Feature Example (VBA)

This example shows how to insert a scale feature.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions  
' 1. Scales the part by a factor of 3.  
' 2. Examine the Immediate window and graphics  
'    area.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swFeatMgr As SldWorks.FeatureManager  
    Dim swScaleFeat As SldWorks.Feature  
    Dim swScale As SldWorks.ScaleFeatureData  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    Set swFeatMgr = swModel.FeatureManager  
    Set swScaleFeat = swFeatMgr.InsertScale(swScaleAboutOrigin, True, 3, 3, 3)  
    If swScaleFeat Is Nothing Then Exit Sub  
  
    Set swScale = swScaleFeat.GetDefinition  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "  Scale = " & swScaleFeat.Name  
    Debug.Print "    Error (0 = no errors) = " & swScaleFeat.GetErrorCode  
  
End Sub
```

Insert Sheet Metal Base Flange Example (C#)

This example shows how to insert a sheet metal base flange.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Creates two boss extrudes and converts them to sheet metal parts.
// 2. Inserts a sheet metal base flange that connects the two sheet metal parts.
// 3. Examine the graphics area and FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        bool boolstatus;
        long longstatus;
        int status;

        public void Main()
        {
            longstatus = (long)swApp.ResetUntitledCount(0, 0, 0);
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref status);
            Part = (ModelDoc2)swApp.ActiveDoc;

            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.07320616684915, 0.04378582530511, 0.008882453015985, false, 0, null, 0);
            Part.ClearSelection2(true);
            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.09520523544121, 0.05740695090967, 0, -0.03844330645187, -0.0429584598942, 0);
            Part.ShowNamedView2("Trimetric", 8);
            Part.ClearSelection2(true);
            object myFeature = null;
            myFeature = Part.FeatureManager.FeatureExtrusion2(true, false, true, 0, 0, 0.01, 0.01, false, false, false, false, 0.01745329251994, 0.0174532
9251994, false, false, false, false, true, true, true, 0, 0, false);
            boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0785775433435, 0.01894373057962, 0, true, 0, null, 0);
            boolstatus = Part.FeatureManager.InsertConvertToSheetMetal(0.002, false, false, 0.004, 0.002, 0, 0.5);
            Part.ClearSelection2(true);

            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.02256810687936, 0.06039039042219, 0, 0.02390260459754, -0.04039198125838, 0);
            Part.ClearSelection2(true);
            myFeature = Part.FeatureManager.FeatureExtrusion2(true, false, true, 0, 0, 0.01, 0.01, false, false, false, false, 0.01745329251994, 0.0174532
9251994, false, false, false, false, true, true, true, 0, 0, false);
            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0009118315510932, 0.02609254832731, 0, true, 0, null, 0);
            boolstatus = Part.FeatureManager.InsertConvertToSheetMetal(0.002, false, false, 0.004, 0.002, 0, 0.5);
            Part.ClearSelection2(true);

            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.ClearSelection2(true);
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05411927414525, 0.01318437124604, 0, -0.007403979976402, -0.001979918613586, 0);
            CustomBendAllowance customBendAllowanceData = null;
            customBendAllowanceData = null;
            myFeature = Part.FeatureManager.InsertSheetMetalBaseFlange2(0.002, false, 0.004, 0.02, 0.01, false, 0, 0, 1, customBendAllowanceData, false, 2
, 0.0001, 0.0001, 0.5, true, false, true, true);
            Part.ClearSelection2(true);
        }
    }
}

public SldWorks swApp;
```

Insert Sheet Metal Base Flange Example (VBA)

This example shows how to insert a sheet metal base flange.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates two boss extrudes and converts them to sheet metal parts.
' 2. Inserts a sheet metal base flange that connects the two sheet metal parts.
' 3. Examine the graphics area and FeatureManager design tree.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As ModelDoc2
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long

Sub main()

    Set swApp = Application.SldWorks
    boolstatus = swApp.ResetUntitledCount(0, 0, 0)
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prt", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set Part = swApp.ActiveDoc

    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.07320616684915, 0.04378582530511, 0.008882453015985, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.09520523544121, 0.05740695090967, 0, -0.03844330645187, -0.0429584598942, 0)
    Part.ShowNamedView2 "Trimetric", 8
    Part.ClearSelection2 True
    Dim myFeature As Object
    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, True, 0, 0, 0.01, 0.01, False, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, False, True, True, 0, 0, False)
    boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0785775433435, 0.01894373057962, 0, True, 0, Nothing, 0)
    boolstatus = Part.FeatureManager.InsertConvertToSheetMetal(0.002, False, False, 0.004, 0.002, 0, 0.5)
    Part.ClearSelection2 True

    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.SketchManager.InsertSketch True
    Part.ClearSelection2 True
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.02256810687936, 0.06039039042219, 0, 0.02390260459754, -0.04039198125838, 0)
    Part.ClearSelection2 True
    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, True, 0, 0, 0.01, 0.01, False, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, False, True, True, 0, 0, False)
    boolstatus = Part.Extension.SelectByID2("", "FACE", 9.118315510932E-04, 0.02609254832731, 0, True, 0, Nothing, 0)
    boolstatus = Part.FeatureManager.InsertConvertToSheetMetal(0.002, False, False, 0.004, 0.002, 0, 0.5)
    Part.ClearSelection2 True

    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05411927414525, 0.01318437124604, 0, -0.007403979976402, -0.001979918613586, 0)
    Dim customBendAllowanceData As Object
    Set myFeature = Part.FeatureManager.InsertSheetMetalBaseFlange2(0.002, False, 0.004, 0.02, 0.01, False, 0, 0, 1, customBendAllowanceData, False, 2, 0.0001, 0.0001, 0.5, True, False, True, True)
    Part.ClearSelection2 True

End Sub
```

Insert Sheet Metal Base Flange Example (VB.NET)

This example shows how to insert a sheet metal base flange.

```

'-----
' Preconditions: Verify that the specified part template exists.
' Postconditions:
' 1. Creates two boss extrudes and converts them to sheet metal parts.
' 2. Inserts a sheet metal base flange that connects the two sheet metal parts.
' 3. Examine the graphics area and FeatureManager design tree.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim boolstatus As Boolean
    Dim longstatus As Long, longwarnings As Long

    Sub main()
        boolstatus = swApp.ResetUntitledCount(0, 0, 0)
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\2016\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, longstatus)
        Part = swApp.ActiveDoc

        Part.SketchManager.InsertSketch(True)
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.07320616684915, 0.04378582530511, 0.008882453015985, False, 0, Nothing, 0)
        Part.ClearSelection2(True)
        Dim vSkLines As Object
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.09520523544121, 0.05740695090967, 0, -0.03844330645187, -0.0429584598942, 0)
        Part.ShowNamedView2("Trimetric", 8)
        Part.ClearSelection2(True)
        Dim myFeature As Object
        myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, True, 0, 0, 0.01, 0.01, False, False, False, False, 0.01745329251994, 0.01745329251
994, False, False, False, False, True, True, True, 0, 0, False)
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0785775433435, 0.01894373057962, 0, True, 0, Nothing, 0)
        boolstatus = Part.FeatureManager.InsertConvertToSheetMetal(0.002, False, False, 0.004, 0.002, 0, 0.5)
        Part.ClearSelection2(True)

        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        Part.SketchManager.InsertSketch(True)
        Part.ClearSelection2(True)
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.02256810687936, 0.06039039042219, 0, 0.02390260459754, -0.04039198125838, 0)
        Part.ClearSelection2(True)
        myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, True, 0, 0, 0.01, 0.01, False, False, False, False, 0.01745329251994, 0.01745329251
994, False, False, False, False, True, True, True, 0, 0, False)
        boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0009118315510932, 0.02609254832731, 0, True, 0, Nothing, 0)
        boolstatus = Part.FeatureManager.InsertConvertToSheetMetal(0.002, False, False, 0.004, 0.002, 0, 0.5)
        Part.ClearSelection2(True)

        Part.SketchManager.InsertSketch(True)
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        Part.ClearSelection2(True)
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05411927414525, 0.01318437124604, 0, -0.007403979976402, -0.001979918613586, 0)
        Dim customBendAllowanceData As Object
        customBendAllowanceData = Nothing
        myFeature = Part.FeatureManager.InsertSheetMetalBaseFlange2(0.002, False, 0.004, 0.02, 0.01, False, 0, 0, 1, customBendAllowanceData, False, 2, 0.
0001, 0.0001, 0.5, True, False, True, True)
        Part.ClearSelection2(True)

    End Sub

    Public swApp As SldWorks

End Class

```

Insert Sheet Metal Edge Flange Example (VBA)

This example shows how to insert a sheet metal edge flange.

```
'-----
'
' Preconditions: A sheet metal part is open and the edge for
'               the flange is selected.
'
' Postconditions: An edge flange is created.
'
'-----

Option Explicit
Dim swApp As SldWorks.SldWorks

Sub main()
    Dim swModel      As SldWorks.ModelDoc2
    Dim bValue       As Boolean
    Dim swEdge       As SldWorks.Edge
    Dim dAngle       As Double
    Dim dLength      As Double
    Dim swFeature     As SldWorks.Feature
    Dim swEntity     As SldWorks.Entity
    Dim swSketch     As SldWorks.Sketch
    Dim vSketchSegments As Variant
    Dim swSketchLine  As SldWorks.SketchLine
    Dim swStartPoint  As SldWorks.SketchPoint
    Dim swEndPoint    As SldWorks.SketchPoint
    Dim nOptions      As SwConst.swInsertEdgeFlangeOptions_e
    Dim dSize         As Double
    Dim dFactor1     As Double
    Dim dFactor2     As Double
    Dim aFlangeEdges(0) As SldWorks.Edge
    Dim vFlangeEdges  As Variant
    Dim aSketchFeats(0) As SldWorks.Sketch
    Dim vSketchFeats  As Variant

    ' Connect to SOLIDWORKS Design
    Set swApp = Application.SldWorks

    ' Get active document
    Set swModel = swApp.ActiveDoc

    '
    ' Flange parameters

    ' Set the angle.
    dAngle = (90# / 180#) * 3.1415926535897

    dLength = 0.01

    ' Position view so SelctByID2 coordinates will lead to a pick
    swModel.ShowNamedView2 ""*Trimetric", -1
    swModel.ViewZoomtofit2

    ' Select edge for flange
    bValue = swModel.Extension.SelectByID2("", "EDGE", 0.007817329387943, 0.001156559535341, 0.04430115457222, False, 0, Nothing, 0)

    ' Get edge
    Set swEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)

    ' Insert a sketch for an Edge Flange
    Set swFeature = swModel.InsertSketchForEdgeFlange(swEdge, dAngle, False)

    ' Select
    bValue = swFeature.Select2(False, 0)

    ' Start sketch editing
```

swModel.**EditSketch**

' Get the active sketch

Set swSketch = swModel.**GetActiveSketch2**

,

' Add the edge to the sketch

' Cast edge to entity

Set swEntity = swEdge

' Select edge

bValue = swEntity.**Select4**(False, Nothing)

' Use the edge in the sketch

bValue = swModel.SketchManager.**SketchUseEdge**(False)

' Get the created sketch line

vSketchSegments = swSketch.**GetSketchSegments**

Set swSketchLine = vSketchSegments(0)

' Get start and end point

Set swStartPoint = swSketchLine.**GetStartPoint2**

Set swEndPoint = swSketchLine.**GetEndPoint2**

,

' Create additional lines to define sketch

,

' Set parameters defining the sketch geometry

dSize = swEndPoint.X - swStartPoint.X

dFactor1 = 0.1

dFactor2 = 1.25

' Add directly and do not display to speed things up

swModel.**SetAddToDB** True

swModel.**SetDisplayWhenAdded** False

swModel.**CreateLine2** swStartPoint.X, swStartPoint.Y, 0#, swStartPoint.X, swStartPoint.Y + dLength, 0#

swModel.**CreateLine2** swStartPoint.X, swStartPoint.Y + dLength, 0#, swStartPoint.X + dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#

swModel.**CreateLine2** swStartPoint.X + dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#, swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#

swModel.**CreateLine2** swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#, swEndPoint.X, swEndPoint.Y + dLength, 0#

swModel.**CreateLine2** swEndPoint.X, swEndPoint.Y, 0#, swEndPoint.X, swEndPoint.Y + dLength, 0#

' Reset

swModel.SetDisplayWhenAdded True

swModel.**SetAddToDB** False

' Commit changes made to the sketch

swModel.**InsertSketch2** True

' Set options

nOptions = swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRadius + swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRelief

#If 0 Then

' Insert the edge flange

Set swFeature = swModel.FeatureManager.**InsertSheetMetalEdgeFlange**(swEdge, swSketch, nOptions, dAngle, 0#, swFlangePositionTypes_e.swFlangePositionTypeBendOutside, dLength, swSheetMetalReliefTypes_e.swSheetMetalReliefNone, 0#, 0#, 0#, swFlangeDimTypes_e.swFlangeDimTypeInnerVirtualSharp, Nothing)

#Else

Set aFlangeEdges(0) = swEdge

Set aSketchFeats(0) = swSketch

vFlangeEdges = aFlangeEdges

vSketchFeats = aSketchFeats

Set swFeature = swModel.FeatureManager.**InsertSheetMetalEdgeFlange2**((vFlangeEdges), (vSketchFeats), nOptions, dAngle, 0#, swFlangePositionTypes_e.swFlangePositionTypeBendOutside, dLength, swSheetMetalReliefTypes_e.swSheetMetalReliefNone, 0#, 0#, 0#, swFlangeDimTypes_e.swFlangeDimTypeInnerVirtualSharp, Nothing)

```
#End If  
End Sub
```

Insert Sheet Metal Gusset Feature Example (C#)

This example shows how to insert a sheet metal gusset feature and modify its data.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\SMGussetAPI.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Inserts four sheet metal gussets.
// 2. Press F5 repeatedly and observe the gusset modifications.
// 3. Inspect the Immediate window for the flatten settings of all gussets.
// 4. Expand Flat-Pattern in the FeatureManager design tree, right-click
//    Flat-Pattern(1), and click Unsuppress.
// 5. Observe the center marks and profiles of all the gussets in their
//    flattened states.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertSMGusset_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;
        Feature myFeature1;
        Feature myFeature2;
        Feature myFeature3;
        Feature swFeat;
        SMGussetFeatureData swFeatData;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID("", "FACE", -0.0538403893476698, 0.0036701308754914, 0.05530817474488, false, 0,
            boolstatus = Part.Extension.SelectByID("", "FACE", -0.0177780871801474, -0.0307393226379986, 0.0341128529187245, true,

            // Gusset #1 insertion parameters
            //1. bOffset = True
            //2. dOffset = 50 mm
            //3. bFlipOffsetSide = False
            //4. profDimType = 0 (indent depth dimensioning scheme)
            //5. dIndentDepth = 10 mm
            //6. dLength = 0
            //7. bUseAngle = False
            //8. dHeight = 0
            //9. dAngle = 0
            //10. bFlipSides = False
            //11. dWidth = 10 mm
            //12. dThickness = 3 mm
            //13. bDraft = True
            //14. dDraftAngle = 3 degrees
            //15. bInnerCornerFillet = True
            //16. dInnerCornerFilletRadius = 2 mm
            //17. bOuterCornerFillet = True
            //18. dOuterCornerFilletRadius = 1 mm
            //19. gussetType = 0 (rounded back)
            //20. bEdgeFillet = False
            //21. dEdgeFilletRadius = 0 mm
            //22. bOverrideDoc = True
            //23. bShowProfile = True
            //24. bShowCenter = True

            myFeature = Part.FeatureManager.InsertSheetMetalGussetFeature3(true, 0.05, false, (int)swSheetMetalGussetProfileDimType_
            0.01, 0.003, true, 3 * 0.0175, true, 0.002, true, 0.001, (int)swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Rour
            0, true, true, true);
            Part.ClearSelection2(true);

            // Gusset #2 insertion parameters (same as for Gusset #1 with the following exceptions)
            //2. dOffset = 30 mm
            //19. gussetType = 1 (flat back)
            //20. bEdgeFillet = True
            //21. dEdgeFilletRadius = 1 mm

            //Select faces
        }
    }
}
```

```

boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0538403893476698, 0.0036701308754914, 0.05530817474488, false, 0,
boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0177780871801474, -0.0307393226379986, 0.0341128529187245, true,

myFeature1 = Part.FeatureManager.InsertSheetMetalGussetFeature3(true, 0.03, false, (int)swSheetMetalGussetProfileDimType
0.01, 0.003, true, 3 * 0.0175, true, 0.002, true, 0.001, (int)swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Flat
0.001, true, true, true);
Part.ClearSelection2(true);

// Gusset #3 insertion parameters (same as for Gusset #1 with the following exceptions)
//2. dOffset = 30 mm
//4. profDimType = 1 (length + height dimensioning scheme)
//5. dIndentDepth = 0 mm
//6. dLength = 25 mm
//7. bUseAngle = False
//8. dHeight = 15 mm
//9. dAngle = 0
//10. bFlipSides = False
//19. gussetType = 1 (flat back)
//20. bEdgeFillet = True
//21. dEdgeFilletRadius = 1 mm

boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0538403893476698, 0.0036701308754914, 0.05530817474488, false, 0,
boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0177780871801474, -0.0307393226379986, 0.0341128529187245, true,

myFeature2 = Part.FeatureManager.InsertSheetMetalGussetFeature3(true, 0.03, false, (int)swSheetMetalGussetProfileDimType
0.02, 0.003, true, 3 * 0.0175, true, 0.002, true, 0.001, (int)swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Flat
0.001, true, true, true);
Part.ClearSelection2(true);

// Gusset #4 insertion parameters (same as for Gusset #1 with the following exceptions)
//2. dOffset = 30 mm
//20. bEdgeFillet = True
//21. dEdgeFilletRadius = 1 mm

//Select orientation and position references
boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0538403893476129, -0.00224553153327633, 0.087801420904043, true,
boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0235965800548001, -0.0307393226379986, 0.0897844682415894, true,
boolstatus = Part.Extension.SelectByID2("Line1@Sketch6", "EXTSKETCHSEGMENT", -0.00609049483400968, -0.0895139047397037,
boolstatus = Part.Extension.SelectByID2("Point1@Sketch7", "EXTSKETCHPOINT", 0.0180407947995604, -0.0762728416981986, 0,

myFeature3 = Part.FeatureManager.InsertSheetMetalGussetFeature3(true, 0.03, false, (int)swSheetMetalGussetProfileDimType
0.01, 0.003, true, 3 * 0.0175, true, 0.002, true, 0.001, (int)swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Rour
0.001, true, true, true);
Part.ClearSelection2(true);

System.Diagnostics.Debugger.Break();

//Five modifications to gusset feature data:

//a. Modify type, draft, and outer corner fillet options for gusset #1
boolstatus = Part.Extension.SelectByID2("Sheet Metal Gusset1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeat = (Feature)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);

swFeatData = (SMGussetFeatureData)swFeat.GetDefinition();
swFeatData.AccessSelections(Part, null);

swFeatData.GussetType = 1; //flat back
swFeatData.DraftSideFaces = false;
swFeatData.FilletOuterCorners = false; //no outer corner fillet

Debug.Print("Sheet Metal Gusset1 Flatten Settings");
Debug.Print(" Override document property settings? " + swFeatData.OverrideDocSettings);
Debug.Print(" Show center marks? " + swFeatData.ShowCenter);
Debug.Print(" Show profile? " + swFeatData.ShowProfile);

swFeat.ModifyDefinition(swFeatData, Part, null);
swFeatData.ReleaseSelectionAccess();

System.Diagnostics.Debugger.Break();

//b. Modify orientation reference of gusset #3
boolstatus = Part.Extension.SelectByID2("Sheet Metal Gusset3", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeat = (Feature)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);

swFeatData = (SMGussetFeatureData)swFeat.GetDefinition();
swFeatData.AccessSelections(Part, null);

boolstatus = Part.Extension.SelectByID2("Line1@Sketch6", "EXTSKETCHSEGMENT", -0.00609049483400968, -0.0895139047397037,
object refLine = null;
refLine = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
swFeatData.ReferenceLine = refLine;

swFeat.ModifyDefinition(swFeatData, Part, null);

System.Diagnostics.Debugger.Break();

//c. Modify legs of gusset #2: select one bend face instead of two flat faces
boolstatus = Part.Extension.SelectByID2("Sheet Metal Gusset2", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeat = (Feature)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);

swFeatData = (SMGussetFeatureData)swFeat.GetDefinition();
swFeatData.AccessSelections(Part, null);

boolstatus = Part.Extension.SelectByID2("", "FACE", 0.03831148650454, -0.0327672470662037, 0.147978181958194, false, 0,

```

```

object newBendFace = null;
Face2[] bendfaces = new Face2[1];
bendfaces[0] = (Face2)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);

newBendFace = bendfaces;
swFeatData.SupportingFaces = newBendFace;
Debug.Print("Sheet Metal Gusset2 Flatten Settings");
Debug.Print(" Override document property settings? " + swFeatData.OverrideDocSettings);
Debug.Print(" Show center marks? " + swFeatData.ShowCenter);
Debug.Print(" Show profile? " + swFeatData.ShowProfile);

swFeat.ModifyDefinition(swFeatData, Part, null);

System.Diagnostics.Debugger.Break();

//d. Modify reference position of gusset #3 - 3 mm away from vertex of hexagonal cut
boolstatus = Part.Extension.SelectByID("Sheet Metal Gusset3", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeat = (Feature)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);

swFeatData = (SMGussetFeatureData)swFeat.GetDefinition();
swFeatData.AccessSelections(Part, null);

boolstatus = Part.Extension.SelectByID("", "VERTEX", -0.0538403893475499, -0.0100654290631334, 0.205954465964501, false

object refPoint = null;
refPoint = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
swFeatData.ReferencePoint = refPoint;
Debug.Print("Sheet Metal Gusset3 Flatten Settings");
Debug.Print(" Override document property settings? " + swFeatData.OverrideDocSettings);
Debug.Print(" Show center marks? " + swFeatData.ShowCenter);
Debug.Print(" Show profile? " + swFeatData.ShowProfile);

swFeat.ModifyDefinition(swFeatData, Part, null);

System.Diagnostics.Debugger.Break();

//e. Modify type and inner corner fillet options for gusset #4
boolstatus = Part.Extension.SelectByID("Sheet Metal Gusset4", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeat = (Feature)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);

swFeatData = (SMGussetFeatureData)swFeat.GetDefinition();
swFeatData.AccessSelections(Part, null);

swFeatData.GussetType = 0; //rounded back
swFeatData.FilletInnerCorners = false; //no inner corner fillet

Debug.Print("Sheet Metal Gusset4 Flatten Settings");
Debug.Print(" Override document property settings? " + swFeatData.OverrideDocSettings);
Debug.Print(" Show center marks? " + swFeatData.ShowCenter);
Debug.Print(" Show profile? " + swFeatData.ShowProfile);

swFeat.ModifyDefinition(swFeatData, Part, null);
swFeatData.ReleaseSelectionAccess();

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```